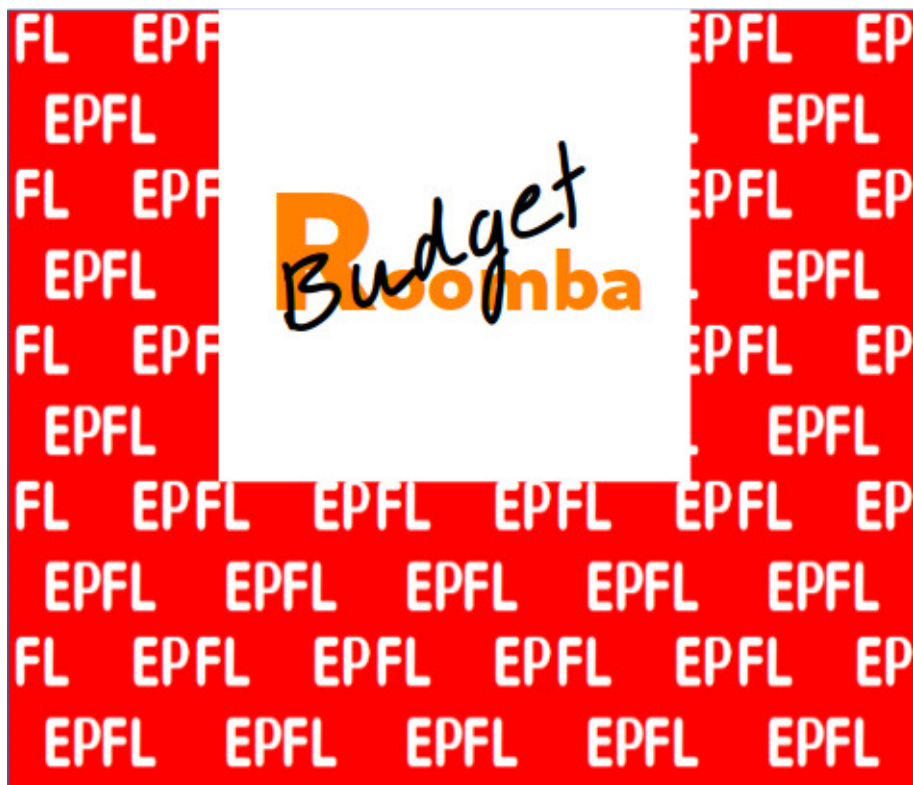




ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

Team 2

BUDGET ROOMBA



STI Robot Competition Report

student 1, Student 2, Student 3

18th June 2023

ACKNOWLEDGEMENTS

We would like to thank the EPFL MAKE initiative to enable this project. Especially the mechanical coach Ivan Tomic was a good technical and moral support and showed great patience in our dozens of screw-orders. Lastly, we appreciate the unmatched devotion of [REDACTED] to his role.

Contents

1	Introduction	4
1.1	Context	4
1.2	Goals	4
2	Design Concept	6
2.1	Localisation	6
2.2	Locomotion	6
2.3	Brick Detection	6
2.4	Collection Mechanism	7
2.5	Compute Hardware	8
2.6	Software	8
2.7	Competition Strategy	9
3	Mechanical Design	10
3.1	Chassis and Drivetrain	10
3.2	Electronics mounting	13
4	Electric and Electronic Design	15
4.1	Material Selection	15
4.1.1	Maxon Motors	15
4.1.2	ESCON Controller and Motherboard	15
4.1.3	Sensors	15
4.1.4	Coral USB AI Accelerator	15
4.1.5	White LED Strip	16
4.2	Connections	16
4.3	Power Consumption	18
5	Software Design	19
5.1	Software	19
5.1.1	Robot Description	20
5.1.2	Drivers	20
5.1.3	Localization	25
5.1.4	Navigation Software	27
5.1.5	Brick Detection	28
5.1.6	Brick Management	30
5.1.7	Main Control	31
5.1.8	Gazebo Simulation	33
5.2	Remarks	34
6	Management	35
6.1	Budget	35

6.2	Time	36
6.3	Work Distribution	38
7	Competition Results	39
7.1	Absolute Performance	39
7.2	Relative Performance	39
8	Discussion and Conclusion	41
9	Appendix	42
9.1	grabbing mechanism ideas	42
9.1.1	Manipulator arm	42
9.1.2	Rotary system 1	43
9.1.3	Rotary system 2	44
9.1.4	Two degrees of freedom gripper	45
9.2	Suction manipulator	46
9.3	Full Node Graph	48
9.4	Crontab Configuration	49
9.5	Custom Camera Calibration	49
9.6	Duplo Detection Training Set	50
9.7	Training Metrics	51
9.8	Custom Nav2 Behavior Tree	52
9.9	Search Grid Waypoints	52
9.10	Purchase History	53
9.11	Gantt Chart of Midterm Presentation	54

be completely autonomous which means that all computations must be performed on the robot.

It has to collect the toys from different zones bring them back to a collection point in one piece. Depending on the area the toys are, it will give more or less points (all zones are shown in Figure 1.1) :

- Collection point 100% : It is the area that we need to collect the duplos in if we want to get all of the points from the toys.
- Collection point 50% : The duplos brought back here are worth half of their points.
- Zone 1 : It is the zone that gives the least amount of points by toys collected but is also the simplest zone to navigate.
- Zone 2 : It gives 2 times the points of the the first zone. It is a carpet with long hairs that is harder to navigate on.
- Zone 3 : This zone is accessible only if we push a button near a door. The toys collected in this arena gives 4 times the points of zone 1.
- Zone 4 : There are 2 ways to go in this zone. The first one is through "B2" which is a ramp and "B3" which are steps. This zone is in our opinion the hardest zone to go in and brings as much points as the 3rd one.

The arena is surrounded by walls and it has 4 light beams at each corner. It is filled with obstacles and the minimal distance between obstacles or between an obstacle and a wall is 50cm. In summary, our robot must be able to:

- Move around in the arena,
- Localize itself in the arena,
- Detect Duplo bricks,
- Collect Duplo bricks,
- Release Duplo bricks in the right area,
- Avoid obstacles

CHAPTER 2

DESIGN CONCEPT

This section summarizes our decision-making process for our robot design. Based on last year's competition observation, we weigh robustness higher than performance - we want to create a robot that achieves its goal almost always even though the overall score might be not so high.

2.1 LOCALISATION

The arena - a static area enclosed with physical borders - is an ideal application for Lidar-based Simultaneous Localization And Mapping (SLAM). To support the localization, we use the low-cost IMU as well as wheel speeds as odometry system.

2.2 LOCOMOTION

For locomotion of small-scale robots, there are multiple options available. We chose the 2 wheel differential drive with castor-wheel very early in the project since it is one of the most widely-used locomotion systems. As a consequence, we can base the control- and navigation-software on well-established algorithms that matured over decades. In particular, we can benefit from the Lidar-system used for state-estimation as it is one of the most common sensor sources for obstacle avoidance in modern robot systems. This locomotion enables extremely high maneuverability such that we can master the navigation even of challenging obstacle parkours.

2.3 BRICK DETECTION

We decided to use computer vision for brick detection. With proper calibration, we can not only estimate *if* there is a brick, but also *where* a brick is. Furthermore, there exists a plethora of high-performant Computer Vision algorithms. In order to perfect this localization, we target to place the camera as high as possible, facing down almost straight. By this, we can estimate the 2D coordinate of a brick whereas we can only compute the direction if the camera faces forward.

Color-Based	⊕ Computational very cheap ⊖ Expected to be very non-robust (transparent bricks and the LEDs in the corners change the perception)
Contour-Based	⊖ High CPU-compute effort ⊖ Expected to be non-robust (as false positives are likely to appear at carpet edges)
Neural-Network	⊕ High robustness expected with proper training ⊖ High compute effort (can be resolved with external hardware accelerator) ⊖ Requires training set

TABLE 2.1
Computer-Vision based Brick-Detection techniques

Since robustness was a clear goal for the design, we selected the Neural-Network based approach. We ordered sufficiently early in the project phase the corresponding AI accelerator in order to run inference on our mobile robot. The dataset-issue was targeted right in the beginning as well by us creating our own training data (see "Software" section).

2.4 COLLECTION MECHANISM

Our initial strategy was to use 2 soft cylinders made of foam to grab the Duplo bricks up a slide, which would then fall into a storing place (see fig. 2.1). The door at the back of the robot would have opened to let the Duplos fall into the dropoff zone.

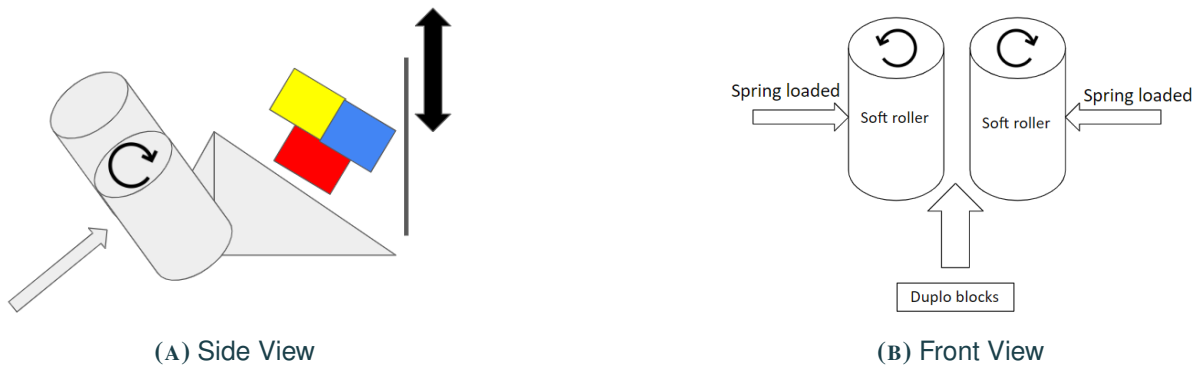


FIGURE 2.1
Initial Collection Mechanism Concept

The rollers would be laterally spring-loaded in order to make them compliant to bigger brick-constructions. After some initial tests, this idea was rejected due to its mechanical complexity in favor of a simpler design. The final design understands the robot mainly as an open box that drags the Duplo constructions over the floor. For easy access, we mount the electronics on top. Furthermore, we discovered that a large frame holding the camera in the bird's eye view can be easily integrated into this approach. Figure 2.2 shows multiple concept drawings of this idea. Tests with bricks on carpet-tiles revealed that neither friction nor blocking of bricks is a big issue as long as the tiles do not overlap each other.



FIGURE 2.2
Final Collection Mechanism Concept

All other ideas for the collection mechanism that we did not consider in-depth are listed in the appendix in section 9.1.

2.5 COMPUTE HARDWARE

We selected the Raspberry Pi 4 (4GB) as main computer for our robot (with addition of the mentioned AI accelerator). This choice was made for 2 reasons

- **Availability:** There are many Pis as well as well-tested battery-based power-supplies in the catalog - this means that ...
 - ... we can obtain replacement in case of failure quickly
 - ... that power supply to the computer is reliable
- **Maker-Community:** Since Raspberry Pis are very wide-spread in the maker-scene, very good documentation and help is expected to be available online

The biggest concern with this selection is compute power especially due to online-SLAM. In case this poses a problem, we would simply add another Raspberry Pi via an Ethernet-Cable to share the computational load. There exists furthermore the option to overclock the Raspberry Pi.

2.6 SOFTWARE

To run the Raspberry Pi, we decided to use Ubuntu 22.04 and the Robot Operating System (ROS) in version 2 in the latest long-term-support (LTS) release "Humble Hawksbill". The choice for ROS is the logical conclusion of our localization and locomotion strategies since it holds many high-performant algorithms as well as visualization tools for our use-case. The version-choice was made as we expect the best support and stability for the current LTS release (i.e. that latest algorithms are available here) and to learn ROS2. ROS is furthermore designed with multi-machine setups in mind such that our fallback-option of using 2 Raspberry Pis in case of compute power shortage could be easily set-up if needed.

2.7 COMPETITION STRATEGY

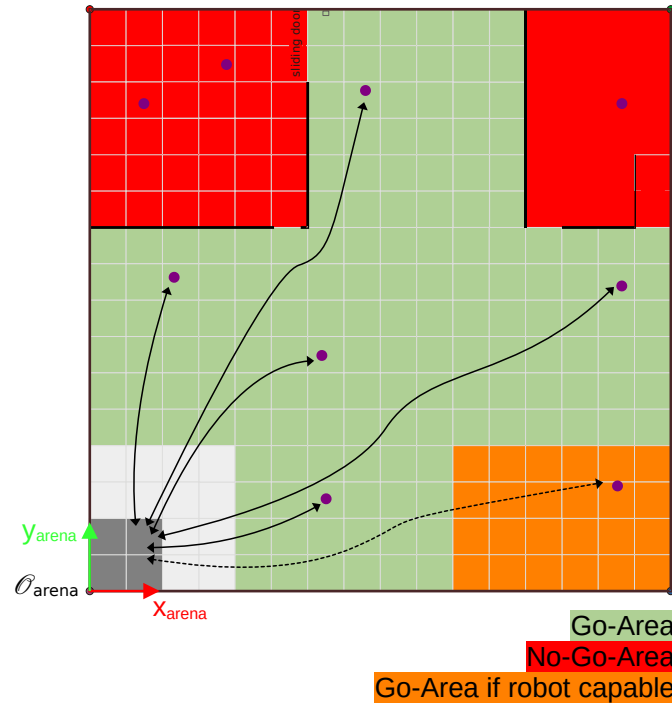


FIGURE 2.3
Strategy of Duplo Collection

Our strategy follows a greedy-paradigm. The robot follows a search pattern and collects a Duplo construction upon detection (including drop-off). In general, we do not target to enter the platform (zone 4) nor the door-protected area (zone 3). We focus on zone 1 and expand our search horizon to the carpet region (zone 2) if our dragging-design works sufficiently well here. This is summarized in fig. 2.3. This strategy does not aim particularly high in terms of expected points. However, we follow our principle of robustness here: If we first collect and only store the bricks in the robot and drop all of them off at the very end, any intermediate failure would make us lose *all* points. With our strategy, we incrementally build up our score and if a failure appears in the middle of the competition, we already have a few bricks dropped off. Also the focus on zone 1 simplifies the mechanical and software design which allows us to optimize the components specifically for this region.

CHAPTER 3

MECHANICAL DESIGN

3.1 CHASSIS AND DRIVETRAIN

In order to have a first prototype available quickly, we opted for a very simple and modular design for the main body by starting with a $300 \times 200 \times 200$ mm box open on the bottom and front. In order for the robot to be as maneuverable as possible, while also keeping the hull empty of components to allow for brick collection and storage, we mounted the wheels directly to the motors, which were themselves mounted entirely outside the hull using 3D printed parts. We also added a heist door lifted using a servo motor in order to enable backwards driving with stored bricks. This design can be seen in figure 3.3a. We also use a single castor wheel in the middle of the backplate of the robot for simplified maneuverability.

While this first prototype was quite functional and enabled us to start working on the electronics and control of the robot, the motor mountings made it very wide (467 mm). Since the **minimal obstacle distance announced in the rules of the competition was 500mm**, we sought to reduce the overall width of the robot, while retaining as big a collection space as possible. Hence, we opted for a belt transmission system, enabling us to keep the same wheel position along the length of the robot while mounting the motors directly to the inner walls of the robot, minimizing the lost storage space by placing them in the back. This choice of placement also brought forth an improvement in stability, as we shifted more of the weight to the back of the robot, greatly reducing the sway of the robot while slowing down. This second generation prototype also discarded the door system, as we were wary of its reliability and decided to use specific navigation software to avoid driving backwards and losing bricks. The design of the second generation can be seen in figure 3.3b

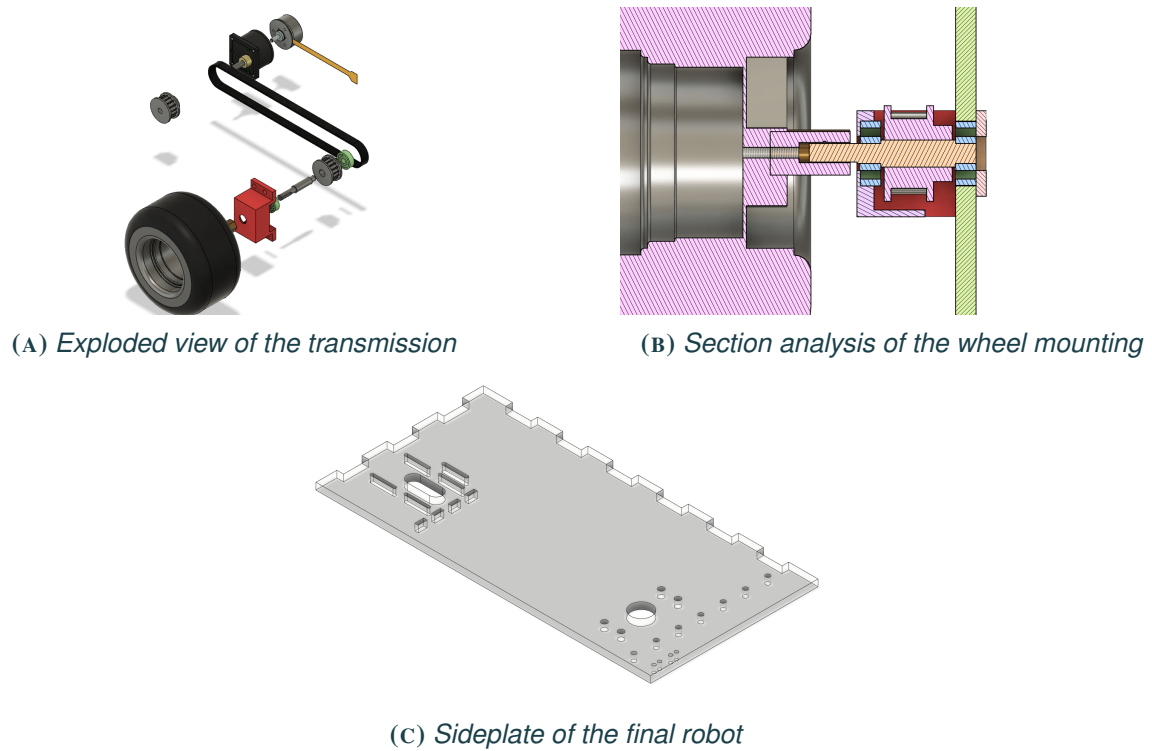


FIGURE 3.1
Transmission implementation

As pictured in figure 3.1a, the transmission is composed of a 475mm long HTD M5 9mm belt, two HTD P15-5M-09F pulleys, two high load ball bearings, a custom made steel axle and 3D printed axle mount. As shown in 3.1b, the bearings are tightly fitted to the wall and the wheel mount piece, and kept in place by stops on the outer edges of the assembly. Furthermore, the belt assembly and tension are enabled thanks to 20mm long slots on the motor side as pictured in figure 3.1c, allowing for large horizontal play during assembly. Implementing this transmission reduced the width of the wheel mountings by 26mm each, for an overall width of **415 mm** instead of the previous 467 mm

To improve the reliability of brick collection and prevent rolling over unrecognized bricks which would greatly disturb the state estimation, we added plates in a funnel fashion. As pictured in figure 3.2, they extend all the way to the maximum width of the robot, starting at a 50 degree angle from the front of the storage space. The black claws were added in the last few modifications of the third generation robot and are made of flexible material.

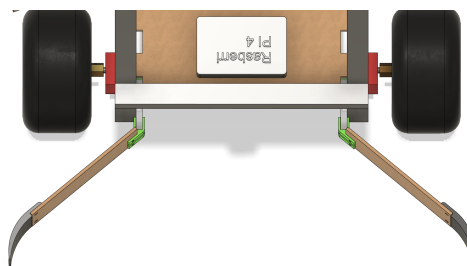
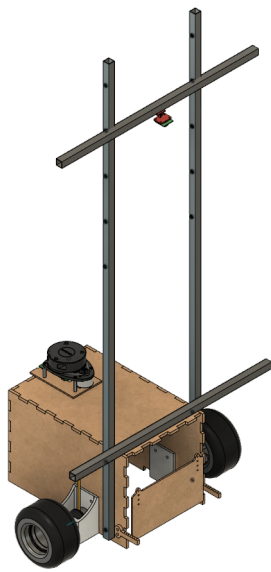
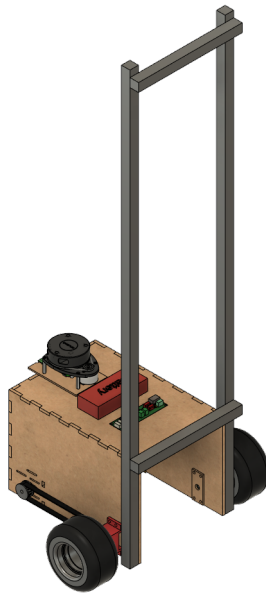


FIGURE 3.2
Funnel-like design

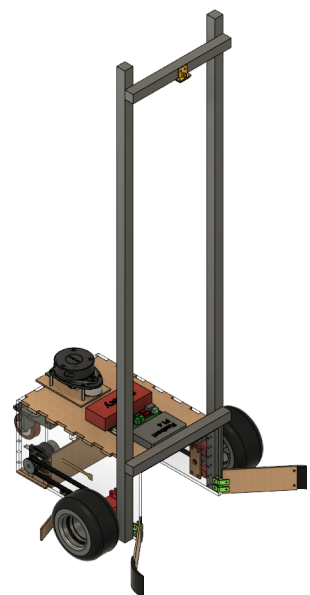
Finally, pictured in figure 3.3c is our 3rd and final robot generation, in which we reduced the dimensions of the hull to a $300 \times 200 \times 150$ box to accommodate for the fact that the arena walls were lower than originally announced and our Lidar was hence placed too high to detect them properly. We took this opportunity to make a few final improvements, such as reducing the ground clearance of our robot and making the sidewalls in Plexiglas instead of MDF as it was too soft and presented risks of the motors slipping in their mounts. We also added a preventive protection plate to separate the motors from the collection space, a bumper behind each wheel to prevent snagging on a brick when driving backwards. The trolley wheel mounting is given in figure 3.5b.



(A) First generation



(B) Second generation



(C) Third generation (Final)

FIGURE 3.3
Evolution of the robot through the generations

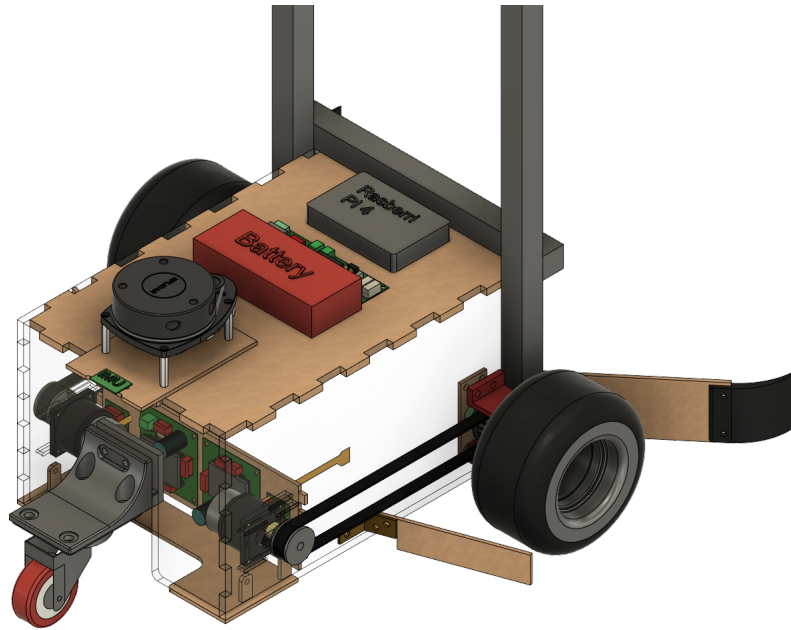
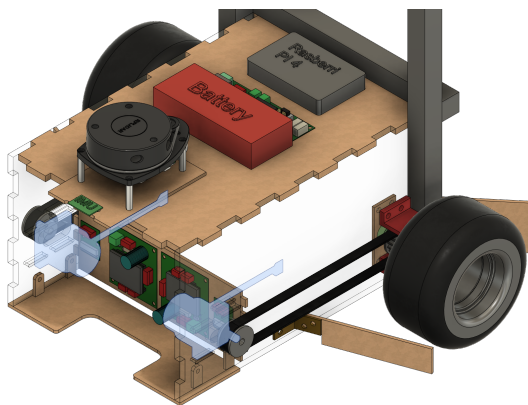


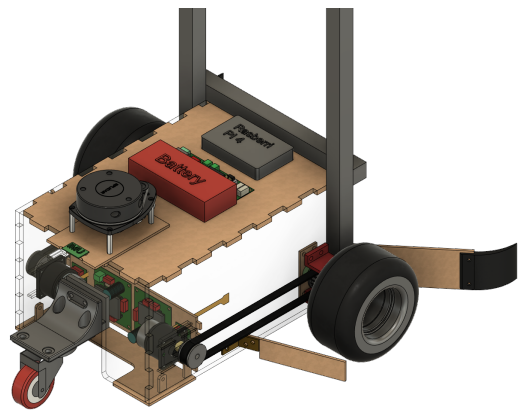
FIGURE 3.4
Back view of the final robot

3.2 ELECTRONICS MOUNTING

The rough placement of the motor control electronics are represented in figure 3.5a. The ESCON motherboards and Maxon adapters are mounted on the motor protective plate, and the powerboard is mounted on top of the box, next to the battery and Raspberry Pi. Pictures of the final robot are compiled in figure 3.6



(A) Placement of the motor control electronics



(B) Big castor wheel positioning

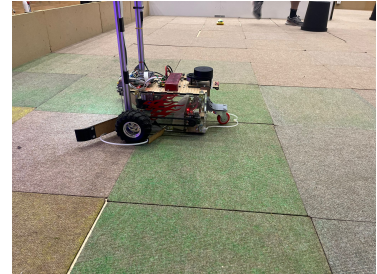
FIGURE 3.5
Back view of the 3rd generation robot



(A)



(B)



(C)

FIGURE 3.6
Impressions of Real Robot

CHAPTER 4

ELECTRIC AND ELECTRONIC DESIGN

4.1 MATERIAL SELECTION

In this chapter we describe the electronics besides the Raspberry Pi that were used to implement our conceptual functionalities.

4.1.1 MAXON MOTORS

Very early in the competition we were advised to use the Maxon motors and decided to go with the one that had the highest reduction gear (the Maxon EC 32 flat (15 W), 1:60 gearbox) since, at the time, we had no substantiated estimate of the torque needed to make our robot move. Therefore we figured that the highest the torque available was, the more headroom we had for developing our robot. The speed of the robot is furthermore not of big concern given that our Online-SLAM dictates the limit for velocity (more explanation in chapter 5).

4.1.2 ESCON CONTROLLER AND MOTHERBOARD

The easiest way to control our motors is to use the "Maxon ESCON 24/2 controller". It is specially developed for Maxon motors and, with the help of the ESCON 24/2 motherboard, fairly easy to use. This was quite a straightforward choice but, we have to clarify that even if the ESCON controllers are easy to use, they are very fragile and we broke 2 of them. The cable of the motors seem to be very fragile as well but luckily, they were fine until the end.

4.1.3 SENSORS

The Lidar we chose is the "RPLIDARA1M8" due its low cost but also due to the availability of plug-and-play drivers. As IMU, we chose the MPU9150 from the catalog. Although it is a low-cost-sensor, tests revealed sufficient performance considering that this sensor only has auxiliary use. As Camera, we chose the "Raspberry Pi Camera v2.1" since it was designed for optimized interplay with the Raspberry Pi's chipset. Furthermore, we use the 1m ribbon cable to connect the camera from the top of its mounting frame. This had no implication of image quality nor latency.

4.1.4 CORAL USB AI ACCELERATOR

In order to handle the brick-detection neural network, we use a hardware accelerator. In particular, we chose the Coral USB AI Accelerator from Google since it promises easy integration into `tensorflow`.

It contains a Tensor Processing Unit (TPU) and the manufacturer claims to accelerate AI model inference by the factor 20 compared to a forward-pass on the vanilla Raspberry Pi.

4.1.5 WHITE LED STRIP

We found a 12V white LED-strip in the "cable-box" that we glued to the front of the robot in order to generate more homogeneous lighting conditions and thus improve brick-detection accuracy.

4.2 CONNECTIONS

The Figure 4.1 shows the electrical schematic of the robot:

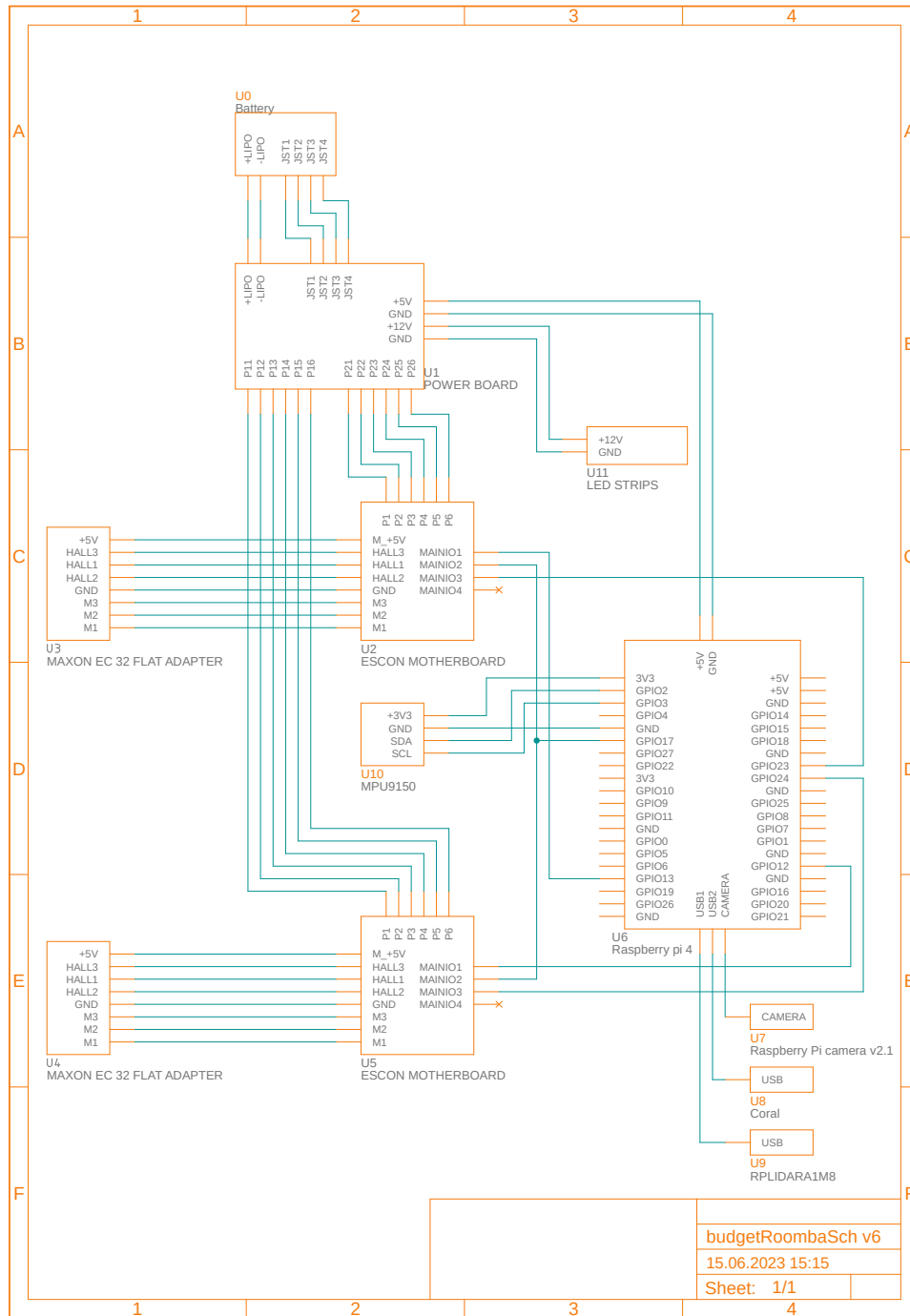


FIGURE 4.1
Schematics

A servo control was implemented on GPIO16 which was used for temporary tests on the first-generation

robot's gate design¹.

4.3 POWER CONSUMPTION

To know approximately the runtime of the robot on a battery charge, we need to know the current consumption during operation. We go through all components and add up their individual draws. This is more an approximate guess and not a detailed value.

The Raspberry Pi consumes under full multithreaded utilization 6W according to [2], i.e. 1.2A at 5V. From [3] we know that the nominal current is 1.06A and set the maximum current to 3A in the ESCON Studio software. The ESCON controllers can withstand 6A for a short period of time according to [4]. This means that ESCON should never break if we load the motors too much. The Coral accelerator draws up to 900mA depending on the load [5]. The power consumption of the camera module is not shown on its datasheet. The estimation found on [6] gives 250mA. The Lidar consumes approximately 350mA according to [7]. When we sum everything and account for a 30% margin of error we obtain²: 6.27-11.31A (depending on the motor load). With our battery capacity of 4000mAh, we can run our robot between 21 to 38 minutes approximately which is enough for one competition run on a fully charged battery. Table 4.1 shows a resume of the current consumption's of the main components.

Maxon motor	1.06 to 3	×2
Coral	0.9	×1
Camera module	0.25	×1
Lidar	0.35	×1
Raspberry Pi 4	1.2	×1
$\Sigma \cdot 1.3$	6.27 to 11.31	

TABLE 4.1

Resume of the current consumption of the main components (in A)

¹We dropped the idea of using servos since we and other teams encountered severe reliability issues in their use

²LED strip is ignored since no specification was found

CHAPTER 5

SOFTWARE DESIGN

5.1 SOFTWARE

The software is designed modularly following the structure of figure 5.1. Each block contains individual programs which communicate via specific protocols. The components of the software can be summarized as follows:

Drivers	Handle low-level communication with hardware components
Brick Detection	Localize Duplo Constructions in RGB Image
Brick Management	Globally register all Duplo detections (acts mainly as detection-memory)
Main Control	Select - based on registered Duplo constructions - if the robot should collect them or instead makes robot follow search grid
Navigation Software	Move robot to goal 2D pose while avoiding obstacles detected via Lidar
Localization	Estimate robot's 2D spatial pose in competition arena and its velocity

TABLE 5.1
Software Architecture Description

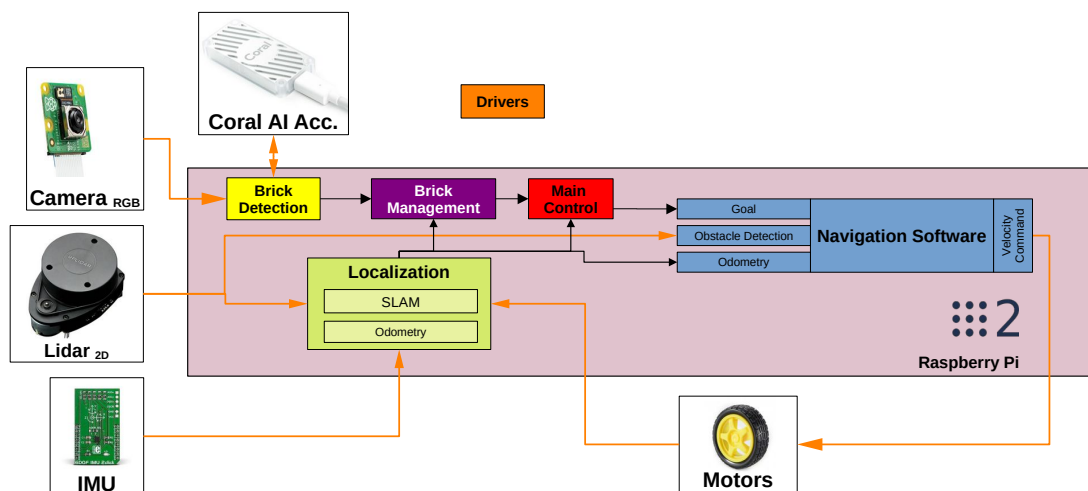


FIGURE 5.1
Software Architecture

In general, we wrote the code in C++ where ever possible for run time performance and stability. For

the following, it is assumed that the reader is familiar with basic ROS vocabulary. The ROS node-graph during full operation is given in appendix 9.3.

5.1.1 ROBOT DESCRIPTION

Corresponding ROS package: `br_description`

In order to organize the sensors' and actuator's relative pose to the robot, ROS's `tf` system is used. Following typical naming schemes in the robot community, we designed the `tf` tree as shown in figure 5.2a (each arrow is a static 6D coordinate transform). The relative transforms were determined from manual measurements and CAD data. Since Lidar and IMU are mounted rigidly on an MDF plate which was maintained throughout the mechanical changes of the chassis, a `nav_unitbase_link` frame is introduced (see fig. 5.2c). That way, we only need to recalibrate the pose of this unit when switching robots (only one step compared to recalibrating Lidar and IMU individually). Following another convention for differential drive robots, we place the root coordinate frame `base_link` in the center of the drive axle projected on the floor as shown in figure 5.2c. The absence of a camera frame is explained below. `main_body` and `back_ball` only serve cosmetic but no functional purpose.

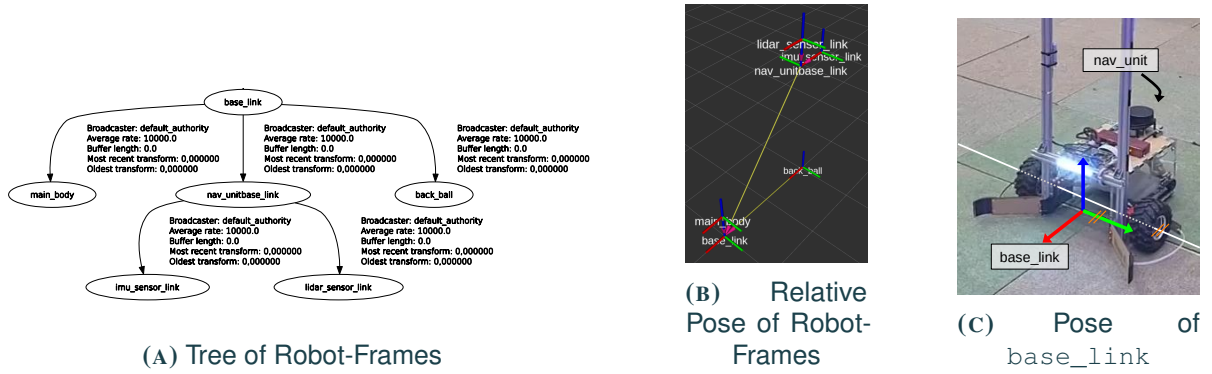


FIGURE 5.2
Robot-`tf` Coordinate Frames

5.1.2 DRIVERS

Corresponding ROS package: `br_drivers`

This package handles the interaction of ROS with the real environment. To permanently grant the drivers low-level hardware access¹, we configured the Cron-Daemon with `sudo crontab -e` and `@reboot` tags such that all required permissions are resolved upon boot (the exact configuration is in appendix 9.4). In order to prevent race-conditions when multiple callbacks within the same program are trying to access the same hardware, we opted for the consequent usage of C++ `std::mutex` locks where ever there is a possibility of this scenario in order to reduce errors.

MAXON MOTORS

Following the ROS convention, the motor driver listens to the `/cmd_vel`-topic of type `geometry_msgs::Twist`. A twist contains in our 2D case the desired linear velocity v_x and angular velocity ω_z (expressed in `base_link` coordinate frame). This driver converts these quantities first into required RPMs of the motors and translates them into meaningful PWM- and motor-spin-direction commands sent to the Raspberry Pi's physical GPIO pins.

¹which is in Ubuntu reserved to the super-user (`sudo`) environment

The RPM computation from the Twist is well-known. The following equations are mainly taken from [8]. The reference frame as well as geometric quantities are described in fig. 5.3:

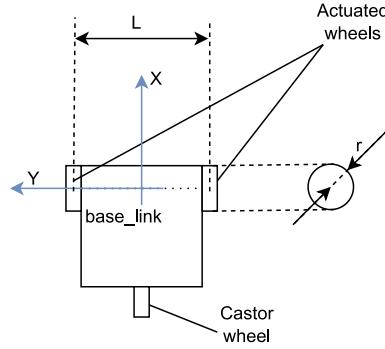


FIGURE 5.3

Reference frame to describe the locomotion of the robot

Given v_x and ω_z , we can compute first the angular velocities of each motor:

$$\omega_{motor,left} = \frac{v_x}{r} - \frac{L\omega_z}{2r} \quad (5.1)$$

$$\omega_{motor,right} = \frac{v_x}{r} + \frac{L\omega_z}{2r} \quad (5.2)$$

Where:

- $\omega_{motor,right}$ = the angular velocity of the right wheel.
- $\omega_{motor,left}$ = the angular velocity of the left wheel.
- r = the radius of the actuated wheels
- L = The length between the two actuator wheels.

Then we can convert them into RPMs like this:

$$RPM_{motor,left} = \frac{\omega_{motor,left} \cdot 60 \cdot G_r}{2 \cdot \pi} \quad (5.3)$$

$$RPM_{motor,right} = \frac{\omega_{motor,right} \cdot 60 \cdot G_r}{2 \cdot \pi} \quad (5.4)$$

Where :

- $RPM_{motor,left}$ = Rotations per minute of the left wheel
- $RPM_{motor,right}$ = Rotations per minute of the right wheel
- G_r = The gear ratio of the motors (which is 60 in our case)

PWMs and the rotation direction is then extracted. The directions are controlled with a simple GPIO digital pin and set via simple `if` clauses. The mapping from RPM to PWM values is a simple linear-affine function that maps one scalar value to another:

$$PWM_{motor,left} = \frac{(RPM_{motor,left} - MIN_{RPM}) \cdot (MAX_{PWM} - MIN_{PWM})}{MAX_{RPM}} + MIN_{PWM} \quad (5.5)$$

$$PWM_{motor,right} = \frac{(RPM_{motor,right} - MIN_{RPM}) \cdot (MAX_{PWM} - MIN_{PWM})}{MAX_{RPM}} + MIN_{PWM} \quad (5.6)$$

Where:

- $PWM_{motor,left}$ = The duty cycle of the PWM value of the left wheel in percentage [%].
 $PWM_{motor,right}$ = The duty cycle of the PWM value of the right wheel in percentage [%].
 MIN_{RPM} = The minimal RPM value ($0min^{-1}$ in our case)
 MAX_{RPM} = The maximal rpm value that we accept ($5000min^{-1}$ in our case)²
 MIN_{PWM} = The minimal PWM duty cycle that the motors accept (10% in our case)
 MAX_{PWM} = The maximal PWM duty cycle that the motors accept (90% in our case)

When performing tests, we saw sometimes that one of the motors was oscillation with high frequency when its RPM was set to $0min^{-1}$. This was due to the poor resolution of the included hall sensors which would output a speed when the motors were not moving resulting in a compensation (since it is a closed loop speed controller) of the motors which would then make them move. This issue caused other mechanical problems like screws of the belt-drive becoming lose. The solution was to set the minimum motor-activation to 100RPM (if it receives a lower command, the motors are not actively controlled).

When configuring the ESCON controllers we could choose that the speed of the motors follows a ramp to prevent the motors from higher currents. We chose *not* to use this feature but to implement our own ramp. This enables us to change the ramp's slope easily without having to recalibrate the motors each time and to know the precise speed of the motors at each time-instant directly. This is possible since we observe that the torque of the motor is more than sufficient to move our robot - If we set an RPM, the robot starts moving directly at the desired speed with very little delay. A ramp is necessary to prevent the robot from tipping if the linear-velocity changes rapidly and to reduce the motors' wear. An illustration of this algorithm is shown in figure 5.4.

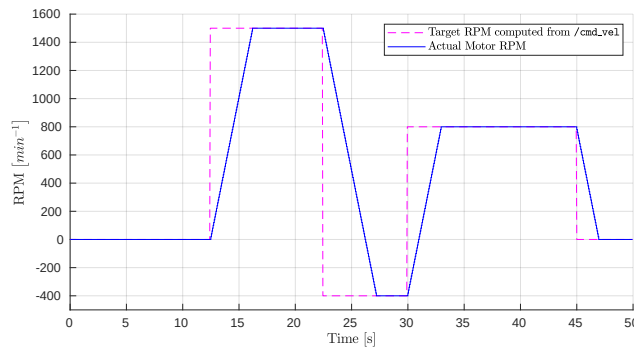


FIGURE 5.4

Example of our Software Ramp for ESCON Motors
(very low increment for better visualization of principle)

The principle of our ramp is straight-forward: The motor driver executes a PWM-update every 50ms. During this update, the driver updates the RPM step-wise towards the desired target velocity from the last message on `/cmd_vel` by a small increment/decrement while the target value is unreached. We also publish the current RPMs as fake-wheel-odometry via a `geometry_msgs::Twist` message every update step. Since the motors' controller follow our software ramp almost perfectly, this odometry estimate is of high quality.

To communicate with the ESCON controllers, the Raspberry Pi has to output for each motor a PWM value to control the speed and a digital value to control the direction. This is handled with `pigpio` framework. This runs a `root` daemon which can be accessed with terminal commands and a C++ library rendering the usage of the output pins straight-forward.

Lastly, we set a routine that enables the motors at start-up of this driver-node. This process consist of

²The maximum possible RPM according to [3] is $10000min^{-1}$ - In our case we put 50% of this value for safety

Sample Frequency	200Hz
Calibration Routine	Once at Start of Driver

TABLE 5.2
IMU Configuration

Sample Frequency	6.67Hz (non-configurable)
Mode	"Sensitivity"
Points per Rotation	8000

TABLE 5.3
Lidar Configuration

outputting a PWM value of 30%, wait for 100ms and output a PWM value of 25% then disable the "enable pin" of both motor controllers, wait for 0.5s and re-enabling them. This routine allows us to start the motors properly from the driver.

MPU9150 IMU

The IMU MPU9150 communicates via the Inter-Integrated Circuit (I^2C) protocol. The driver foundation was written by ChatGPT given the following prompt:

“Write a c++ node for ros2 that publishes IMU data from the MPU9150 chip that is connected to the Computer via I2C. The node should first initialize the IMU and also publish magnetometer and temperature data.”

We could verify with the manuals provided by the manufacturer and by compiling the code that the hardware and register addresses were correctly identified by the chatbot. To populate the outgoing ROS-messages of type `sensor_msgs::Imu` with covariance data, we use the values given in the datasheet in a hard-coded fashion. This is critical to use the data with a Kalman filter. Another addition is an angular-velocity-to-quaternion integrator following [9] to get an impression of the angular drift.

A general problem with IMU data is the presence of bias (i.e. non-zero angular velocities and linear accelerations without motion). In order to mitigate this effect, we implemented a calibration routine that estimates this bias as the arithmetic mean of a 5s horizon and subtracts this from future measurements.

We removed the temperature and magnetometer functionality since good calibration for the latter requires rotating the robot around all 3 axes by at least 360° and since we expect tremendous disturbance from inter alia the aluminium frame and the proximity of the motors.

The configuration used in the final competition is summarized in table 5.2. The frequency was set to the maximum rate working stably on the Raspberry Pi³;

RPLIDAR A1M8

Through the availability of prebuilt open-source drivers suitable for our ROS version, integration was plug-and-play. From the lidar modes, we selected "Sensitivity" in order to scan 8000 points per rotation. The configuration is summarized in table 5.3. A widespread issue with Lidars is self-perception (e.g., we perceive the bars of the camera-frame with the Lidar). To prevent subsequent errors, we set the Lidar minimum range consequently to 30cm in all Lidar-data-consumers.

³Verified via `ros2 topic hz /imu/data`

RASPBERRY PI CAMERA

To use the Raspberry Pi's proprietary camera, two options are available:

- Legacy Camera Stack
 - ⊕ Easy integration into Video4Linux-Framework and therefore easy access through OpenCV
 - ⊖ Very limited Configuration Capabilities of camera video stream
- libcamera
 - ⊕ Official State-of-the-Art library with full access to camera configuration parameters
 - ⊖ Limited support on Ubuntu 22.04 (as library was designed for Raspberry Pi OS)
 - ⊖ No official support for OpenCV

Due to ease of implementation, the Legacy Camera Stack was implemented firstly. However, the field-of-view (FOV) was considerably constrained. Research found that this is due to the non-configurable camera resolution. Hence, libcamera was used instead. Through recompilation of this library on the Pi from source following their instructions [10], we could install it on our platform. We found a third-party library called LCCV [11] enabling OpenCV-interaction with the camera. Through this, we were able to set the camera's sampling resolution to exploit the full FOV and also to configure the sampling frequency.

We calibrated the camera in a specific way such that each pixel (u, v) can be mapped with a linear-affine transform to coordinates of the `base_link` frame (x_{bl}, y_{bl}, z_{bl}) . This mapping assumes that all imaged real-world coordinates of the pixels are in the ground-plane (i.e. the $z_{bl} \equiv 0m$), so it is a $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ mapping at its core. The transform is of 2 steps: First, we rectify the image and warp it into an artificial "birds eye" (denoted with $'$). Through special alignment, the u' -axis of the transformed image is parallel to the x -axis of `base_link` and the v' -axis of the image parallel to the y -axis of `base_link`. Finally, x_{bl} is a standard linear-affine function in u' and vice versa for y_{bl} . The pipeline and formulae are visualized in figure 5.5.

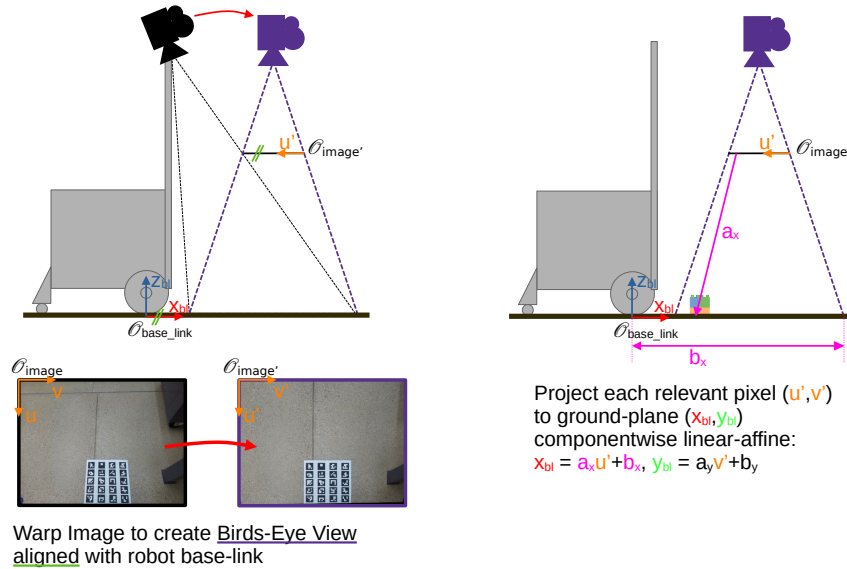


FIGURE 5.5
Principle of Camera Calibration

Although we do not perform 'classic' camera calibration (e.g., distortion compensation, ...), we obtained accuracies in the low cm-domain with this technique. The calibration routine is explained in appendix 9.5. With the camera placement and this image transform in place, the Pi can observe a spot of about

Sample Frequency	10Hz
Sample Resolution	1640×1232
Downsampled Resolution	640×480

TABLE 5.4
Camera Configuration

1.5×1.1m in front of the robot (see fig. 5.6b⁴). One concern is that the bricks can be directly in front of the robot when they get stuck in the fangs while collecting. As this might causing invalid detections, it was resolved by simply placing a black bar on the critical region of the image (see fig. 5.6c).

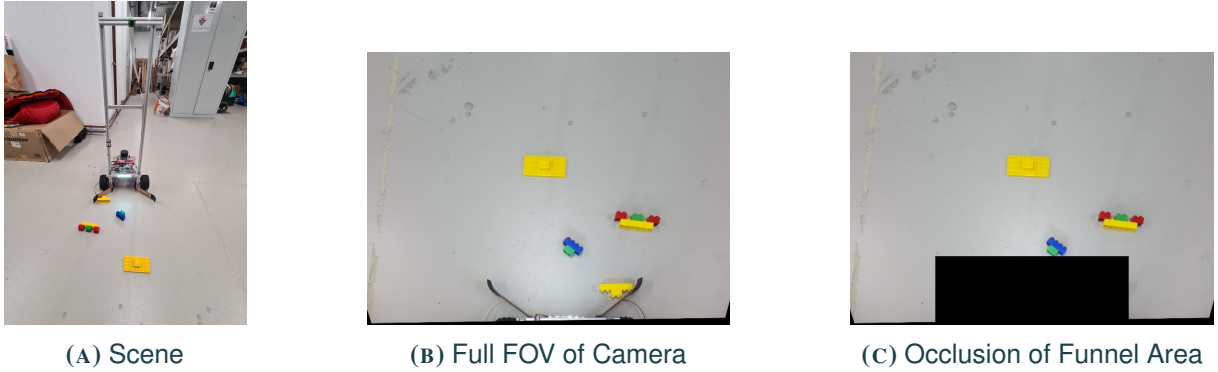


FIGURE 5.6
Camera Capture Example

Finally, we publish the *transformed* image to the ROS system (i.e. with the above calibration applied). The configuration is given in table 5.4. We set the sampling resolution to the full resolution to maximize the FOV but downsample the image later. This downsampling factor as well as the sampling frequency were chosen in compliance with the AI brick detection.

CORAL USB AI ACCELERATOR

There are 2 official drivers available to enable the accelerator - standard ("std") and performance ("max") [12]. The latter pushes the stick to its limits at the cost of significant heat development. Since we mount the Coral directly to the aluminium frame, we could cool it passively and sufficiently for this mode. The Python package `tf_lite` to make `tensorflow` communicate with the accelerator was not available for the Raspberry Pi's chipset at our Ubuntu/Python version. We had to compile the package our self by following the instructions in [13]. This built the Python package wheel with a docker image. We configured this image to be Ubuntu 22.04 with Python 3.10 and selected "aarch64" as target architecture.

5.1.3 LOCALIZATION

Corresponding ROS package: `br_state_estimation`

The localization follows a 2-stage approach commonly found in mobile robots with a high-frequency Odometry system and a low-frequency corrective global referencing system. The latter system is in our case an online Lidar-based SLAM. The `tf` tree can be expanded such that the root is a coordinate frame *fixed to the real world*. This root frame is called `arena` in our application and is placed at the corner of the dropoff-zone of the arena such that the x and y axes align with the arena-edges and the z -axis points up.

⁴As reference, a Duplo-Unit is $\approx 1.6\text{cm}$ - the purely yellow construction in fig. 5.6b on a 4×8 plate is therefore $\approx 6.4\times 12.8\text{cm}$

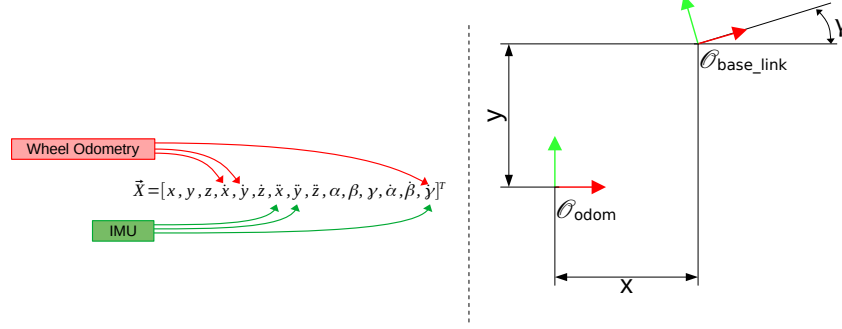


FIGURE 5.7
Considered Quantities in Kalman Filter

ODOMETRY

To estimate the `base_link`'s speed and approximate position in an odometry coordinate frame (`odom`), an Extended Kalman Filter is used to fuse wheel speed and IMU data. We use the prebuilt filter in ROS-package `robot_localization` [14]. The main configuration concerns the processing of sensor data. The internal state of the Kalman filter is 9 dimensional and we set the sensors to affect them according to figure 5.7. Note that the states z , α and β are always 0 in our 2D case. Furthermore, we set the inclusion of IMU data to `differential` mode - meaning that only the differences $\Delta \vec{a}_t = \vec{a}_t - \vec{a}_{t-1}$, $\Delta \vec{\omega}_t = \vec{\omega}_t - \vec{\omega}_{t-1}$ are fused into the estimate in order to mitigate the bias effect of the IMU.

Since direct wheel-speed data is unavailable, we had to artificially generate them. We are successfully using the commands sent to the Maxon motors in a *feed-forward* fashion. A custom ROS-node receives the current command and translates it into a measurement of type `nav_msgs::Odometry`. We attribute a static covariance value to this measurement which we estimated during iterative testing and tuning. Besides completing the `tf` tree, this filter also generates a good estimate of the current robot's velocity which is used by the navigation software.

ONLINE SLAM

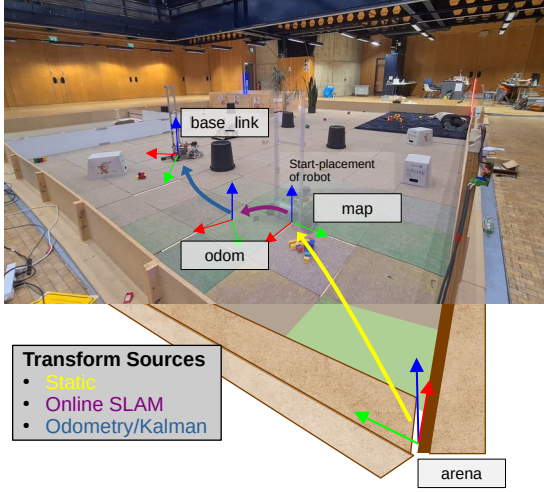
Since the Odometry is subject to drift over time (due to wheel slip and IMU noise), a global corrective algorithm is used. We employ an online Lidar-based SLAM for which multiple algorithms are available in ROS. We selected Google's `cartographer_ros` (based on [15]) due to its availability on our platform and the possibility to use additional sensor sources other than Lidar to improve the SLAM quality. Especially the use of an odometry-estimate as a prior turned out to be very helpful in smoothing the robot-tracking in particular when the robot stands still or rotates on spot. The SLAM-pose is computed in a map frame which is placed at the starting pose of the robot and fixed to the world. We therefore place the robot's `base_link` always at $x = 1m, y = 1m, \gamma = 180^\circ$ w.r.t arena at the start of the competition⁵ and add a `ROS-static_transform_publisher` which provides the arena-to-map transform corresponding to this placement. See figure 5.8a for an illustration of all mentioned frames and the robot start-placement.

RESULT

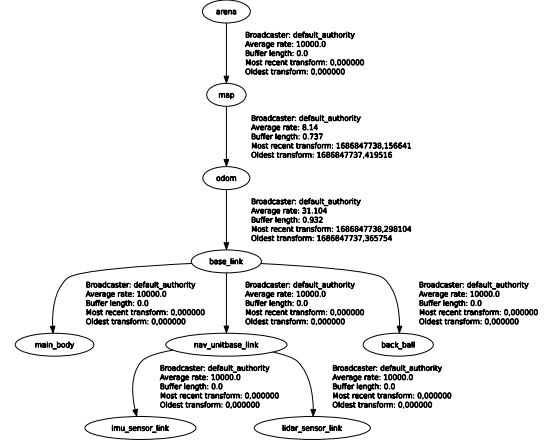
With this approach, we can complete the `tf` tree as shown in figure 5.8b. We observed that especially turning motion is critical whereas pure translative motion does not pose a big challenge for the state-estimation system. We identified $0.25 \frac{rad}{s}$ as maximum turning velocity for which we could estimate

⁵That way, the robot has to turn at the beginning and the SLAM performs a complete sweep of the arena

the state sufficiently well. The Odometry system usually worked well and drifted only up to about 1m over one competition run. The global state-estimation was accurate up to about 15cm over the whole competition run.



(A) Visualization of Ex-Robot tf Tree



(B) Complete tf Tree

FIGURE 5.8
State Estimation Frames

5.1.4 NAVIGATION SOFTWARE

Corresponding ROS package: `br_navigation`

The ROS2 navigation stack `Nav2` [16] is used for collision-free motion control. To summarize, this is a 2-stage approach with a global ("Planner") and local ("Controller") navigation system. Both operate on different grid-based obstaclemaps with cost-values ("Costmaps") that are fed with Lidar-data. While the Planner considers the whole arena and always computes a coarse global path from the current robot-pose to the final goal query, the Controller tries mainly to follow this path on a short-horizon basis by generating motor commands. The Controller usually considers dynamic constraints (like acceleration limits) and performs a sophisticated obstacle avoidance. The Controller's map is therefore of higher resolution as the global map but also smaller due to the short horizon and follows the robot (i.e. is a 'rolling window'). We follow the common practice to artificially inflate the Costmaps to create safety margins through high navigation costs close to the obstacles. To prevent the robot from entering invalid zones - especially the carpet which is not perceived by the Lidar - a static map is added to the local and global costmap. This marks the illegal zones statically as "obstacle". A simulation example of the global mapping is given in figure 5.9. The global path is green, the Controller's path is red (and very small).

The core of `Nav2` however is a Behavior Tree. This determines mainly if the current path-following is successful and if not, what recovery actions must be taken. Since the default tree is more a showcase of `Nav2` capabilities, we tailored our own tree with minimal recovery behaviors in order to increase execution speed since most of the default actions assume dynamic environments. The tree is given in appendix 9.8 but understanding requires advanced `Nav2` and Behavior Tree knowledge.

A summary of the `Nav2` configuration is given in table 5.5. We set the accuracies in general rather low in order to avoid corrective movements and since the robot is equipped by a 30cm-funnel to compensate for errors. The global costmap is padded by 50cm to compensate for worst-case state-estimation inaccuracies. The Regulated Pure Pursuit Controller [17] essentially follows a point on the edge of its horizon on the interpolated global path by continuously computing an arc from the robot's position to this point. If the robot's orientation is too divergent, it is stopped and turned until the headings match properly. A key

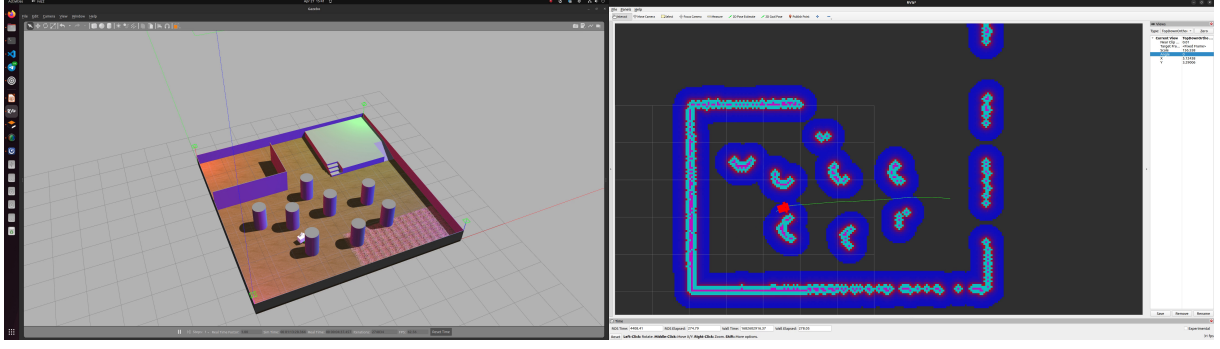


FIGURE 5.9
Simulation Example for Navigation with Paths and Global Costmap

Parameter	Global Planner	Local Controller
Costmap Size	$9 \times 9\text{m}$	$3 \times 3\text{m}$
Costmap Resolution	5cm	2cm
Pathfinding Algorithm	A*	Regulated Pure Pursuit
Pathfinding Frequency	0.5Hz	5Hz
Pathfinding Accuracy	0.15m	0.1m/0.1rad
Horizon	∞	0.7m

TABLE 5.5
Nav2 Configuration

element that makes our strategy work is that this Controller **never drives backwards**, i.e. collected bricks never leave the robot with Nav2 control⁶. The dynamic constraints have been set to $0.3 \frac{\text{m}}{\text{s}}$ for linear velocity and $0.25 \frac{\text{rad}}{\text{s}}$ for angular velocity. The angular limit is set by the state-estimation system and the linear limit by the brick-detection system in order to prevent motion-blur.

5.1.5 BRICK DETECTION

To detect the bricks, we use a Neural-Network based approach.

MODEL SELECTION

As simultaneous detection and localization of the detection are necessary to steer the robot to a Duplo construction, we chose the YOLO architecture as it is high-performant and well-researched. In particular, we selected the YOLOv5 implementation from Ultralytics [18]. The main motivation here is not performance but practical reasons since they provide easy interfaces for training and exporting of models and have a very well maintained GitHub repository. From the available YOLOv5 architectures, we chose the low-parameter "YOLOv5-S" structure for performance.

DATASET GENERATION

Since Duplo datasets for YOLO-training could not be found, we created our own. As a key ingredient to training is the amount and diversity of data, we chose not to manually annotate given images but use a computer-graphics based process to efficiently render artificial images with perfect annotations.

The input to this algorithm are STL files of Duplo bricks and the LEGO-brick color palette. The first was created with the SCAD tool "Doblo Factory" from [19]. The bricks are of size $k \times 2$ with

⁶For unloading of bricks, we bypass this control manually

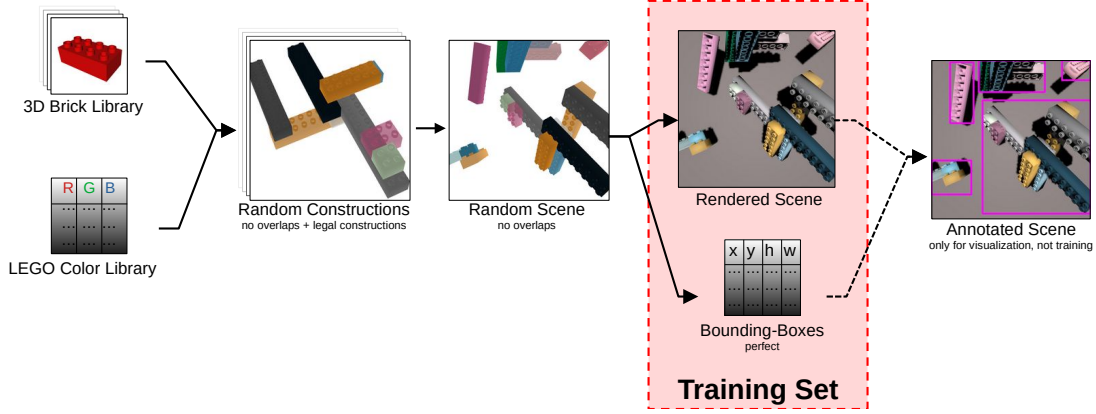


FIGURE 5.10
Rendering-Based Dataset Generation Pipeline

$k \in \{2, 3, 4, 6, 8, 10\}$. The color palette containing transparency information and the color HEX-values was retrieved in csv format from a LEGO fan page [20]. Based on them, the algorithm assembles randomly legal Duplo constructions. "Legal" means that there are no overlapping bricks, and that bricks are only stacked on top of each other in a way that the nibbles of the lower brick are in the corresponding socket of the higher brick. The randomness of this assembly affects the height of the construction, the amount of bricks per layer, the color of the bricks, and the size of the bricks. Those constructions are then placed in a non-colliding way in a virtual rendering scene. In this scene, the camera is placed on top of the bricks facing downwards - just as with the real robot. The ground-plane is either a uniform color or a texture image. Since the camera parameters are perfectly known, we can transform the extends of each construction (i.e. the bounding boxes) to the image plane by using the camera matrices. Finally, we use `pyrender` and specific surface properties to generate realistic samples. The pipeline is visualized in figure 5.10. The texture images as well as adversarial samples (i.e. scenes with non-Duplo items) were obtained from the website "pexels.com" by searching for "texture" and "living room" respectively and downloading the first few hundred hits with the Firefox-Webbrowser-plugin "Download All Images" by Joe Ertaba. Each adversarial image from this website was integrated multiple times in the database with color changes, noise-additions and rotations.

With this procedure, we obtained a dataset with 8500 samples (for details, see appendix 9.6).

TRAINING AND POST-PROCESSING

The training was already implemented by [18]. The key quantities of the training are summarized in appendix 9.7. The trained model must be recompiled for the Coral AI USB accelerator which is conveniently handled fully by the selected YOLO library. This recompilation also reduced the input image size to 320×320 pixels to match the Coral's capabilities.

BRICK REGISTRATION AND INTEGRATION INTO ROS

Since the camera-image is of dimension 640×480 but the input-size of the Coral AI inference is 320×320 , we use a sliding window approach. Spaced by 160 pixels, we crop the image into 6 patches of size 320×320 and pass each through the network (see fig. 5.11a). We then merge adjacent bounding-boxes from all the 6 forward-passes with standard OpenCV operations to generate the final predictions. The processing result of the example scene of fig. 5.6a is given in fig. 5.11b. In this image, the cyan rectangles represent the bounding-boxes found in individual patches and the magenta rectangles correspond to the merge. With the parameters a_x, a_y, b_x, b_y from fig. 5.5 known, the bounding boxes can be translated into `base_link` coordinates. To express the bounding boxes in ROS, we chose `visualization_msgs::Marker`

since they can encode besides the position also simple shapes of an object. We selected an ellipse as shape with the position as center of the bounding-box and x, y -extends as scaled (factor 3) height-and-width of the bounding box. The z -extent corresponds to the YOLO confidence of the detection. The application to the demo-scene is visualized in fig. 5.11c.

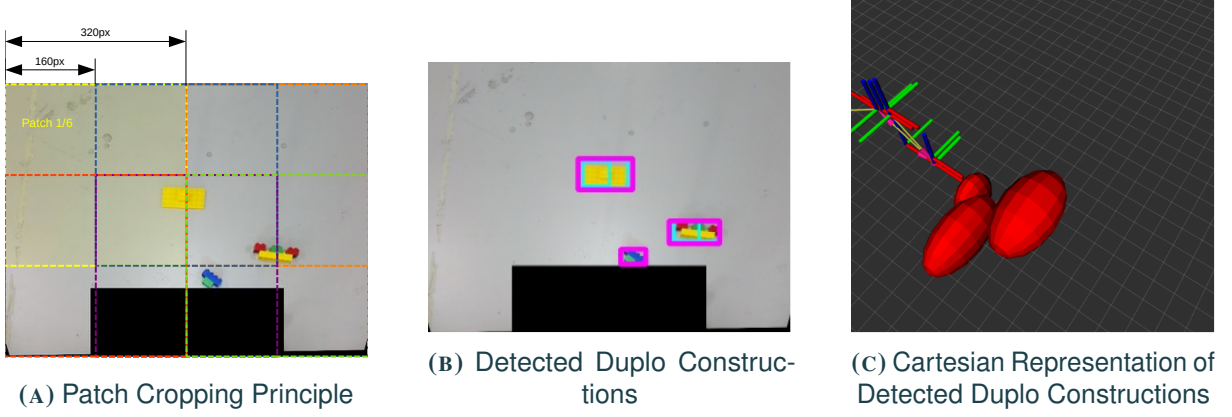


FIGURE 5.11
Duplo Detection

5.1.6 BRICK MANAGEMENT

Corresponding ROS package: `br_brick_management`

In order to globally track all brick detections, a sophisticated brick-mapping-system was invented. The system is built on ETHZ's `grid_map` C++ library [21]. Our algorithm's gridmap contains 2 layers - "bricks" and "valid" - and covers the whole arena with a size of 9×9 m and a resolution of $5 \frac{cm}{cell}$. The origin of the map is the `arena` coordinate frame. If the system is in state `ACTIVE`, Duplo constructions from the brick-detection system are placed in the "bricks" layer with a cosine-kernel⁷. The kernel's position is derived by traversing the `tf` tree and it's size by the x and y size parameters of the corresponding marker. The height of the kernel corresponds to the marker's z -extent ($\leftrightarrow$ confidence). Note that the orientation of the kernel is also considered. If the cell with the largest value exceeds a threshold, a brick-detection interrupt containing this position is forwarded to the main control algorithm (see next section) and the state is set to `PASSIVE`. The state can be reset to `ACTIVE` by the main control algorithm. The gridmap is processed with 5Hz (i.e. the algorithm searches every 0.2s for the maximum). During this process, all values in the grid-map are reduced by a certain factor which suppresses false positive detections that only occur in one frame. The "bricks" layer corresponding to the example scene of fig. 5.6a is given in fig. 5.12.

⁷Instead of Gaussian since cosine is 0 at a specific argument and avoids discontinuities

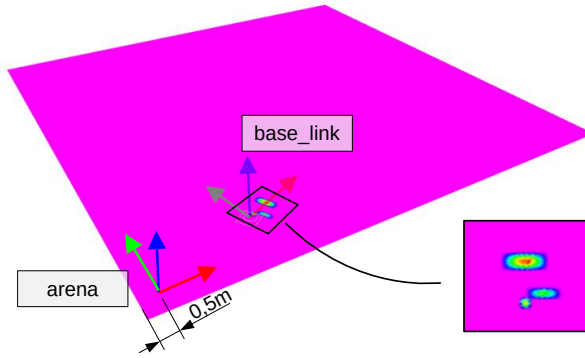


FIGURE 5.12
Illustration of "bricks" Layer

Since the robot cannot collect bricks close to obstacles and to prevent the collection of the same construction twice, detection-markers are pre-validated. This is implemented with the "valid" layer which listens to Nav2's Global Costmap and keeps track of past detections. Detection-Markers are only inserted into the "bricks"-layer if they could be validated in the "valid"-layer.

5.1.7 MAIN CONTROL

Corresponding ROS package: `br_brain`

To coordinate all explained modules, there exists a main control algorithm. This program is connected to the Nav2 stack via 2 interfaces:

- `ComputePathThroughPoses`: Check for given list of poses if there is a path connecting them
- `FollowWaypoints`: Navigate through given list of poses and stop if a pose cannot be reached

The algorithm has 2 states - `SEARCHING` and `COLLECTING`. At beginning, the state is `SEARCHING`. In this state, the algorithm makes the robot follow a given search grid through the arena utilizing the `FollowWaypoints` interface. This grid is given in figure 5.13; the exact waypoint coordinates in appendix 9.9. For safety reasons, this path is extra far away from the carpet region. Once the search grid is done, we restart it⁸. If the `FollowWaypoints` communicates that a waypoint in the search-grid cannot be reached, it is skipped. If the algorithm obtains a brick-detection interrupt from the Brick-Management with the position of a Duplo construction, it firstly checks if it can be reached via the `ComputePathThroughPoses` interface⁹. This is done by querying a 2-element list: $[\vec{x}_{detection}, (0.75m, 0.75m, -135^\circ)]$ (`base_link` target coordinates expressed in the `arena` frame). The last point is in the 100% zone. If negative, we stay on the search grid and set the Brick-Management to `ACTIVE`¹⁰. If positive, it changes its state to `COLLECTING` and collects the detected Duplo construction by sending the identical 2-element-list to the `FollowWaypoints` interface. After that, we manually send a velocity command to the motors to back-off 60cm in order to unload the construction. This bypasses the Nav2 stack which never drives the robot backwards in our configuration. Finally, the robot returns to the search-grid and the program's state is `SEARCHING` again. We furthermore restore the Brick-Management to `ACTIVE`. If the collection process fails at any point, the robot returns to the search grid.

The state evolution is illustrated in fig. 5.14.

⁸In tests, the grid was never completed even once within 10 minutes

⁹It has been pre-validated by the "valid" layer of the Brick-Management but can still be in a closed region

¹⁰Note that the Brick-Management will ignore this construction from now on since it has been registered in the "valid" layer

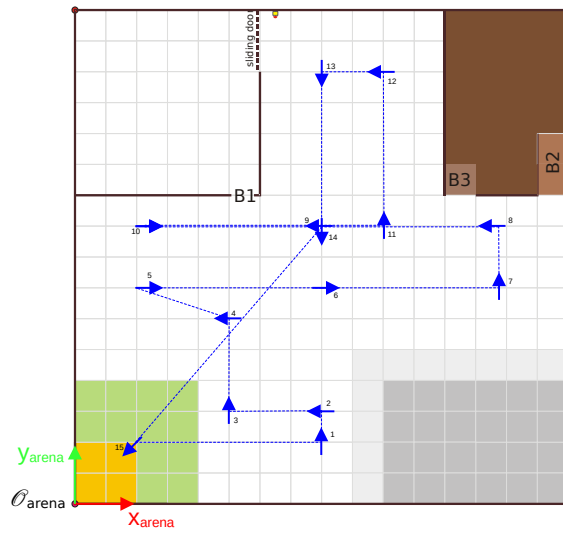


FIGURE 5.13
Search Grid

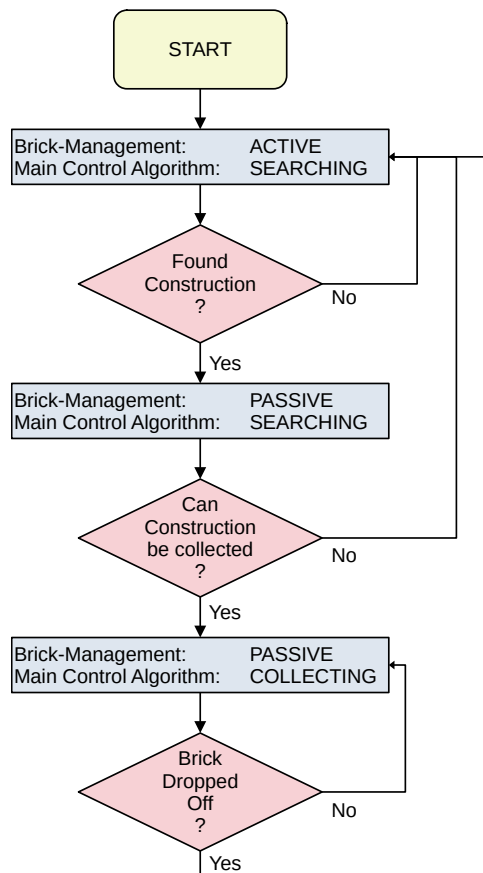


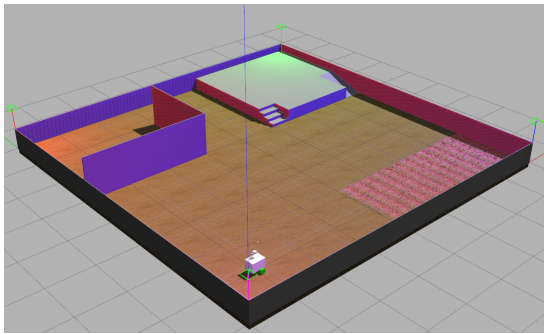
FIGURE 5.14
State Machine

5.1.8 GAZEBO SIMULATION

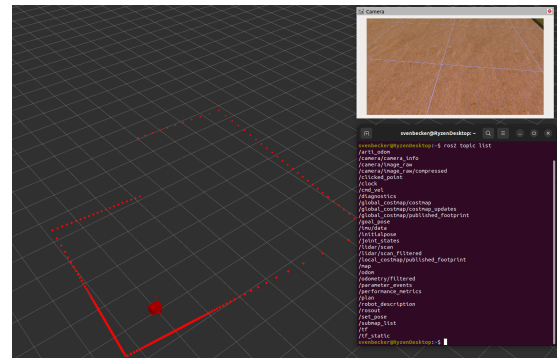
Corresponding ROS packages: `br_simulation`, `arena_gazebo`

An important accelerator of software development was in all project phases the availability of a simulation framework. We chose Gazebo due to its easy ROS integration. The simulator consists of two main components (as shown in fig. 5.15a):

- Model of Arena with ...
 - Approximation of walls and elevations (completely designed by us)
 - Carpet with different friction and compromised visibility (designed by us with grass model found online)
 - 4 orientation lights in the corners
- Model of Robot with ...
 - Sensors: IMU, Lidar, Camera (usage of Gazebo’s off-the-shelf plugins)
 - Actuators: Differential Drive (usage of Gazebo’s off-the-shelf plugins)
 - Castor-Wheel approximation at the back (link with no friction)
 - Optional: Ground Truth Pose Publishing (through Gazebo plugin wrote by us)



(A) View of Arena with Robot



(B) ROS Integration

FIGURE 5.15
Gazebo Simulation

The actuators and sensors are sending and receiving messages from/to ROS through the same protocols as the actual hardware drivers in `br_drivers` (see in fig. 5.15b a visualization of the Lidar, the camera feed, and the advertised topics). The robot model is a coarse approximation of the real robot (in terms of dynamics, extents, and sensor placements) and more a test-bench for setting up the navigation stack, the state-estimate, and the main state-machines. As this was available in the second week of the semester, ROS packages could be preconfigured without the physical robot being ready. After the correct ROS-topic-remapping, state-sequence, ... could be verified in the simulation with very short test cycles, we only had to fine-tune them on the real robot. We also tried hardware-in-the-loop tests by running the Gazebo on a workstation PC and the robot control software on the Raspberry Pi in the same network, however the network latency especially for Lidar- and image-data rendered this approach unsuccessful¹¹.

¹¹We used a local WiFi 802.11ac hotspot to connect both entities *outside of EPFL*

5.2 REMARKS

- The Raspberry Pi (with Coral TPU) can handle the complete software explained in this chapter
- The Raspberry Pi is overclocked to 2GHz and *never* crashed due to active cooling

CHAPTER 6

MANAGEMENT

6.1 BUDGET

For the virtual budget, our purchasing strategy was very aggressive since unused parts may be returned. A full overview of the purchases is given in figure 9.11 of the appendix. In the end, we did not use 8 items from the catalog. Due to various reasons mentioned in this figure, 6 items broke and are hence moved to the real budget. The total spending on the virtual catalog is summarized in table 6.1. Note that screws, laser-cutting, and additively manufactured parts obtained at the EPFL SPOT are not considered in this balance.

Catalog Purchases	+	918.75
Unused Items	-	152.82
Broken Items	-	263.70
		502.23

TABLE 6.1
Virtual Budget Purchase Summary (in CHF)

In contrast, we spent the real budget conservatively. Due to the danger of global supply shortages, we bought the Coral AI accelerator and the Lidar very early in the project since we were sure about their use. The belt-system as well as the Alu-Profiles were purchased in later project phases based inter alia on our experience with the first generation robot. Those purchases were made in online stores according to figure 9.12 in the appendix. We also purchased smaller items local hardware stores (see table 9.9 in the appendix). The record of our real-budget-spending is summarized in table 6.2.

Online Purchases	+	390.86
Local Hardware Store Purchases	+	38.00
Broken Items from Catalog	+	263.70
		692.56

TABLE 6.2
Real Budget Purchase Summary (in CHF)

In the end, we had a massive buffer left in the virtual budget but the real budget was used up to 92%. However, about one third of the latter spending was due to broken parts from the catalog. We note that there was a concatenation going on once: The power supply broke, and this in turn fried our Raspberry

Pi. Since no Pi was left in stock, the referee allowed us to use our own private Raspberry Pi (of same specifications) in the competition. Also, some of the newly purchased parts like the Coral AI-Stick or the Lidar are in general good additions to the catalog and we are certain that future competition teams will benefit from them.

6.2 TIME

In general, our team met on Wednesdays during the Semester to discuss the state of the project and to assemble parts. Apart from this, team-members spent a considerable time on their own to design parts in CAD, write source code, etc. However, due to a lack of a coach, time management was slightly unstructured until the first meeting on March 29th. Until then, we mainly focused on concept-analysis and first software implementations. After that, the team became much more devoted to the project and started to properly design, assemble, and test prototypes - within one week, we had the first generation robot ready and could test robot software. This acted as a base-line on which we incrementally improved our overall designs (software and mechanical) in the following weeks until the competition. Due to the simplicity of our design, we were able to take the learnings from the rehearsal competition into account to design, manufacture, and assemble the third generation robot within 36h 6 days before the competition. Facing this deadline, we had no 'emergency situation' - our robot performed already comparatively good in the rehearsal competition (score: 2.5, 3rd place compared to other teams) and we mainly had to tune it to improve performance in the last few days.

In general, we would rate our time-management very good *after* we met our coach.

All project steps and work-packages are summarized in the Gantt-Chart of figure 6.1.

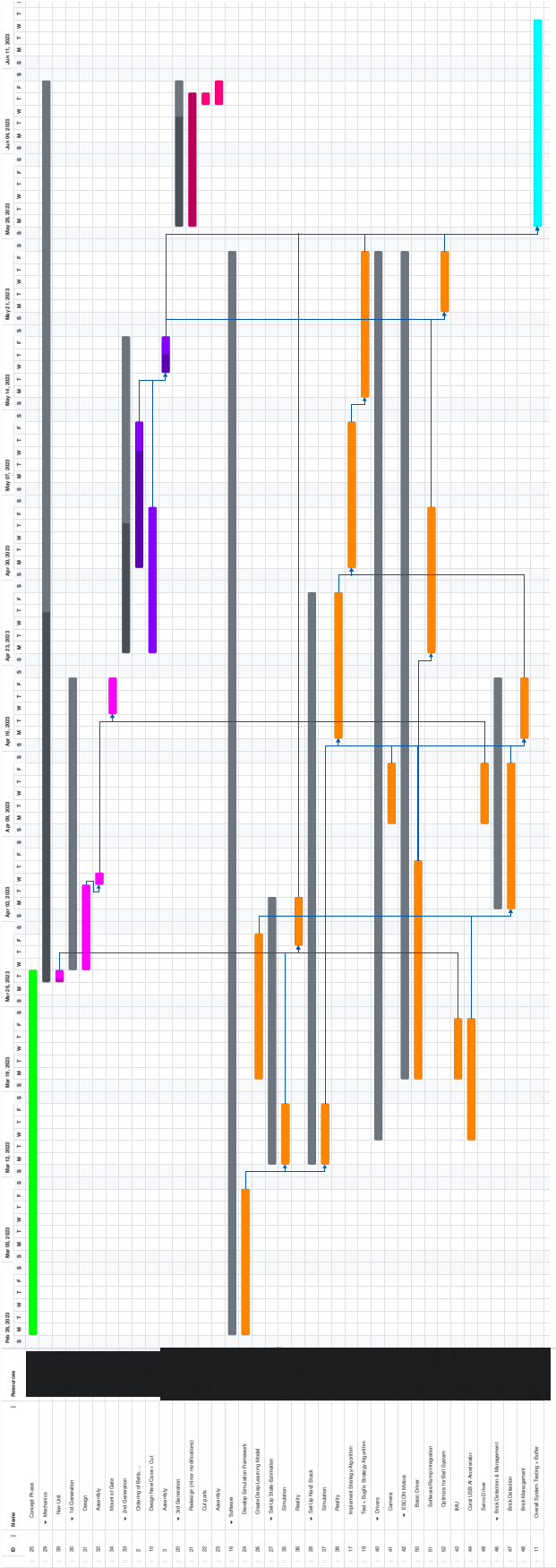


FIGURE 6.1
Gantt Chart
37

6.3 WORK DISTRIBUTION

In the beginning we decided to split the workload into the three main pillars of a robotic project shown in table 6.3 based on our skills:

Field	Member Name
Software	student 1
Mechanics	student 2
Electronics	student 3

TABLE 6.3
Subject split between team members

However, despite the split all team members helped others when it was needed. We did not have all the same background resulting in team members working more than others. It is the case of student 1, who had already industrial robot experience before and demonstrated a higher level of engagement than the other team members.

The assignment of tasks to team members is given in the Gantt chart of figure 6.1. The chart shown at midterm presentation is in appendix 9.13.

CHAPTER 7

COMPETITION RESULTS

In this section, we review the performance of our robot in the real competition.

7.1 ABSOLUTE PERFORMANCE

Our robot was able to achieve our design goal. We were able to collect 7 bricks in total, including all 3 bonus bricks yielding a score of **147.5**. We placed those bricks right next to easy-to-reach standard constructions such that bonus- and standard-bricks could be collected in one run (i.e. we performed the dropoff-sequence 4 times in total). Furthermore, our robot avoided all obstacles and the no-go-zones (like the carpet) and never encountered a false positive brick collection. Due to a slight drift in the state-estimation, only the first few bricks were dropped at the 100% zone with the rest on the edge to the 50% zone which were counted with the lower multiplier by decision of the referee.

7.2 RELATIVE PERFORMANCE

Although our expectations were met during the competition, our robot was outperformed by all other robots¹ in terms of points. The other teams were on the one hand able to drop-off their bricks consistently on the 100% zone but also collected bricks in the high-point zones 2 and 3. Although our collection rate in zone 1 was quite high in comparison, the lucrative other zones ultimately were the key reason for our low relative performance. However, one has to note that according to Prof. Auke Ijspeert, this was "the best competition ever" (in terms of scored points). Given this context and for example the fact that last year, only a few teams were able to collect a minimum amount of items, we are nevertheless quite happy with the outcome. Furthermore, we want to point out that our robot performed partially better in terms of obstacle avoidance than other high-scoring teams. For example one of the Ultra-Sound sensor based teams managed to move *both* potted plants by a few cm during the competition whereas our robot navigated with a sufficient safety margin all the time.

¹Except for the robots that were clearly unfinished

Rank	Team No.	Score
1	4	345
2	1	320
3	6	285
4	2	147.5
5	3	0
6	5	N/A

TABLE 7.1
Result of the Competition

CHAPTER 8

DISCUSSION AND CONCLUSION

In conclusion we went through the explanation of all the design phase of such a robot, from preliminary strategies to advanced software explanations. We were able to build a robot that full-filled the requirements for the competition and explained some of the issues we had regarding the implementation.

Although our expectations set in the concept phase were fully achieved in the competition, they were too conservative. Other teams with more daring strategies had in general better performance. However, our robot was able to deliver similar results during many test-runs in different obstacle and brick layouts as well as in the competition, so our robustness goal was achieved. A major contribution to this is the overall simplicity of the mechanical design which enabled us to have first hardware prototypes available in early project phases and also to redesign some elements with learnings from the rehearsal competition in short time. In general, we consider our management after the first meeting with the coach quite success full - we could present a fully matured robot at the competition that did exactly what we expected it to do. Another functional concern is that we focused too much on obstacle avoidance - both in software (safety margins) and mechanics (slimness of robot) in order to always meet the 50cm constraint set in the rulebook. During the competition, all obstacles were placed further away and higher scoring teams touched them sometimes.

To summarize, in a future competition, we would not stress as strongly on sophisticated obstacle avoidance and set higher expectations to the robot in terms of point-outcome. However, we recommend our "robustness" design philosophy in order to avoid surprises at the competition day.

Lastly, we could expand our skillset through this project software-wise by learning ROS2 and its packages, and by making use of many manufacturing possibilities at SPOT. There are not many projects at EPFL that allow students to build something from scratch and experience all the problems that could arise during the implementation of such an endeavour. For that we are grateful that this project exists and happy that we could participated in it.

CHAPTER 9

APPENDIX

9.1 GRABBING MECHANISM IDEAS

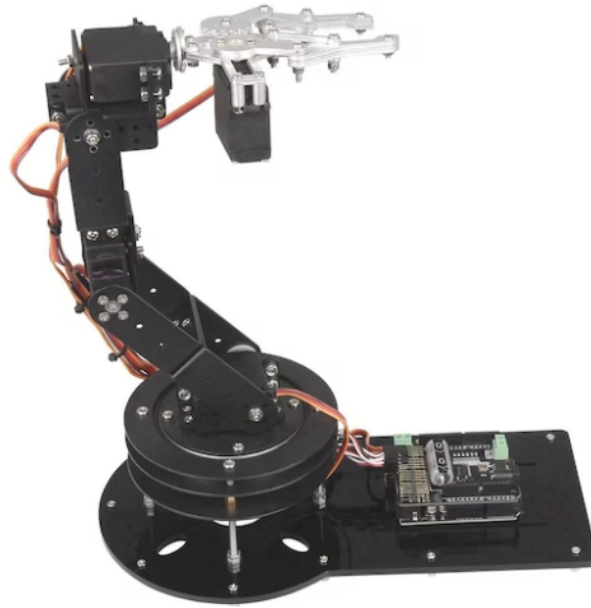
We also had lots of other ideas for the grabbing mechanism which are listed in the following subsections. We have used our own scoring system (0-5) which have the following meaning and that we thought were relevant for the competition :

- Robustness with respect to (abbreviated w.r.t) brick position : It means the ability of the mechanism to comply to different brick positions. The main question to ask for this criterion is how well can the robot grab the bricks if they are in different positions/orientations.
- Robustness w.r.t brick constructions : Similarly to robustness w.r.t brick position, it is how well the robot handles different constructions.
- Robustness w.r.t reliability : It is simply how confident are we that when the robot grabs a brick construction, it doesn't fall from its hand.
- Mechanical complexity : How complex would it be to build it.
- Software complexity : How complex would it be to implement a software that uses the actuators of the grabbing mechanism.
- estimated cost is self explanatory.
- additionnal comments : If we need to explain something in more details

Of course all of these criterion are subjective to our understanding of how we would have implemented the different mechanism. It is not supposed to be the ground truth but simply help us, in the beginning of the competition to have a better clue on what options we have.

9.1.1 MANIPULATOR ARM

One of the first ideas was to use a classical manipulator arm. Of course this is too complex, not very efficient and unreliable so this idea was abandoned really fast.

**FIGURE 9.1**

An example of a robotic arm we could have used taken from [22]

Title	Manipulator
Description	It is a robotic arm with a grabbing end effector (see Figure 9.1)
# of actuators	3-5
Robustness w.r.t brick position	1 (arm must be placed w.r.t to prism position)
Robustness w.r.t brick constructions	1 (Not clear how/where to grab construction)
Robustness w.r.t reliability	3 (if grip force sufficient and object recognition good, bricks can fall out of gripper due to jerk motion)
Mechanical complexity	2 (arm must only be screwed together and placement on the robot might not be easy due to mass distribution)
Software complexity	5 (computer vision must compute exact pose of brick or construction and find best grabbing spot)
Speed	2 (3-step process: grab -> move -> release; movement is rather slow)
Estimated cost	200CHF
Additional comments	-

TABLE 9.1
Manipulator Arm idea

9.1.2 ROTARY SYSTEM 1

Next comes the "shovel" idea, which is represented in Figure 9.2 which features a rotating wheel and a stopper for the movement of the shovel, and a ramp which leads the toys into the storing of the robot. This idea was pretty convincing at first but we thought that the first idea described in section 2.4 was simpler and as explained we finally chose to go in an even simpler solution.

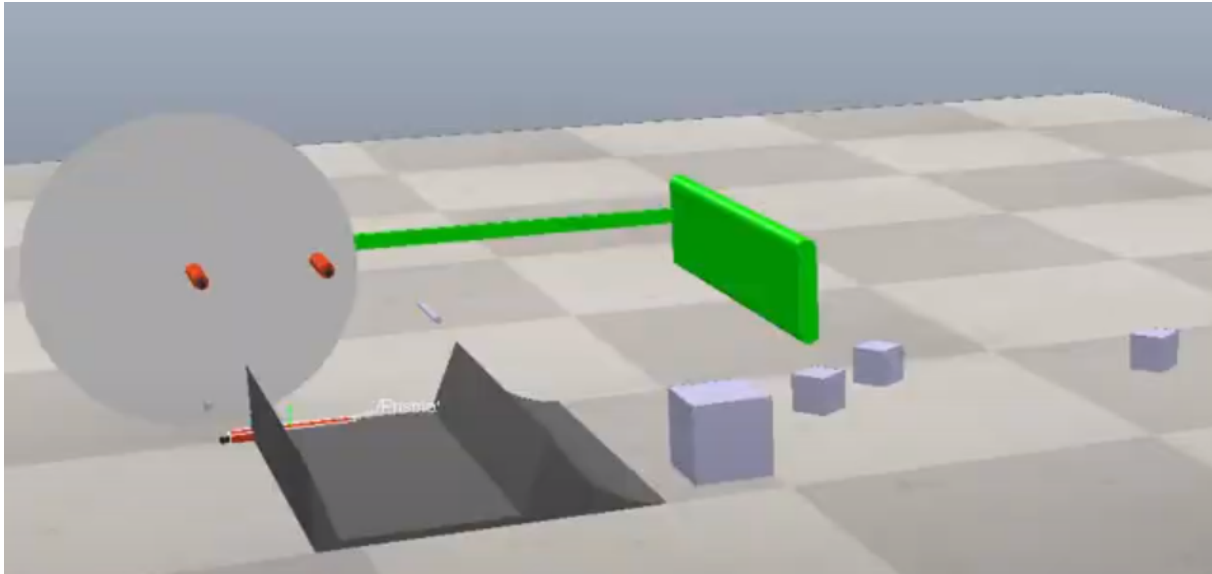


FIGURE 9.2
Rotary system number 1

Title	Rotary system 1
Description	It is a shovel that is moved by a wheel and a stopper to perform the shoveling motion to bring the duplos into the robot
# of actuators	1
Robustness w.r.t brick position	5 (any brick in area in front of robot is taken)
Robustness w.r.t brick constructions	3 (critical if construction is high tower, can break during fall)
Robustness w.r.t reliability	3 (big problem: Jamming when pulling on ramp [they should be flush], should not break easily, fault detection not too complicated)
Mechanical complexity	2 (main concern are the geometrical relations [e.g. disc radius,...])
Software complexity	1 (is simple on-off system)
Speed	4 (depends on motor speed which has effect on reliability)
Estimated cost	50CHF
Additional comments	Useful addition could be movable ramp with servo to make ground-robot-transition flush

TABLE 9.2
Rotary "shovel" system idea

9.1.3 ROTARY SYSTEM 2

Next comes a more exotic kind of grabbing mechanism, which is a mill that elevates the bricks and at the top, the angle of the mill would be too steep and the duplos would fall into the storing area. This mechanism is shown in Figure 9.3. With this system comes a lot of issues that could arise and this is not very compliant to high duplo construction.

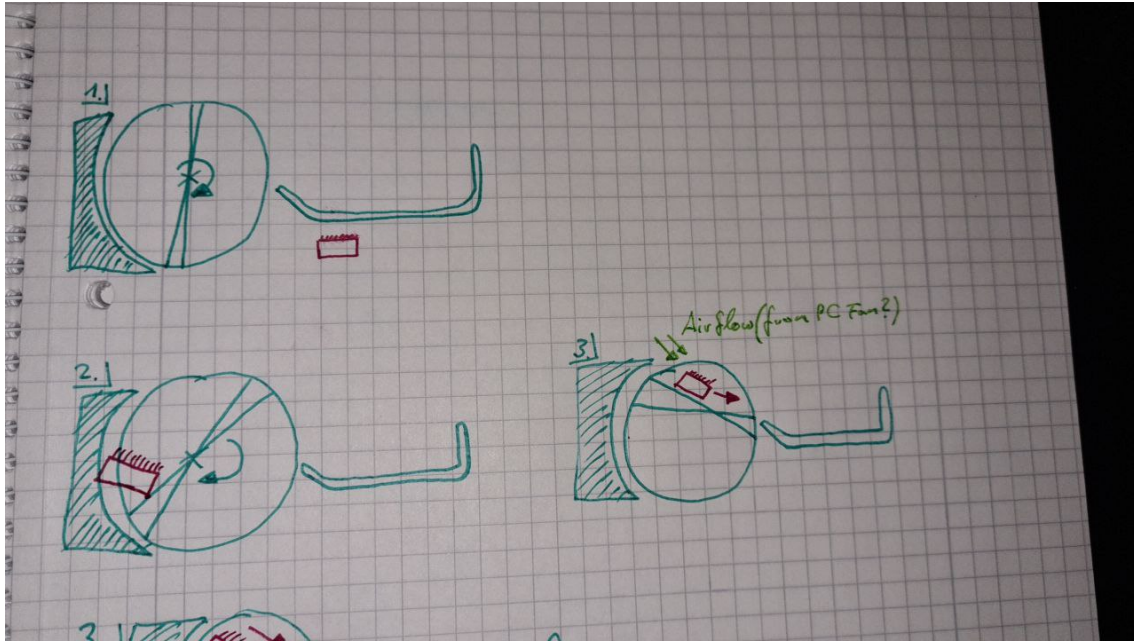


FIGURE 9.3
Rotary system number 2 : The mill

Title	Rotary system 2
Description	Mill elevates brick then at a certain angle bricks overcome friction and fall into storage.
# of actuators	1
Robustness w.r.t brick position	5 (any brick in area in front of robot is taken)
Robustness w.r.t brick constructions	1 (only very low objects allowed, constructions can break in mill)
Robustness w.r.t reliability	2 (big problem: Jamming when pulling on ramp [they should be flush], should not break easily, fault detection not too complicated)
Mechanical complexity	2 (only very low objects allowed, constructions can break in mill)
Software complexity	1 (mill can run continuously)
Speed	5 (robot does not need to stop)
Estimated cost	50CHF
Additional comments	Airstream from (PC) fan can be used to blow bricks from mill to storage

TABLE 9.3
Rotary system 2, "the mill" idea

9.1.4 TWO DEGREES OF FREEDOM GRIPPER

Next we thought of a 2 degrees of freedom gripper as shown in Figure 9.4. Which is less complex than the manipulator arm but still requires precise placement of the gripper.

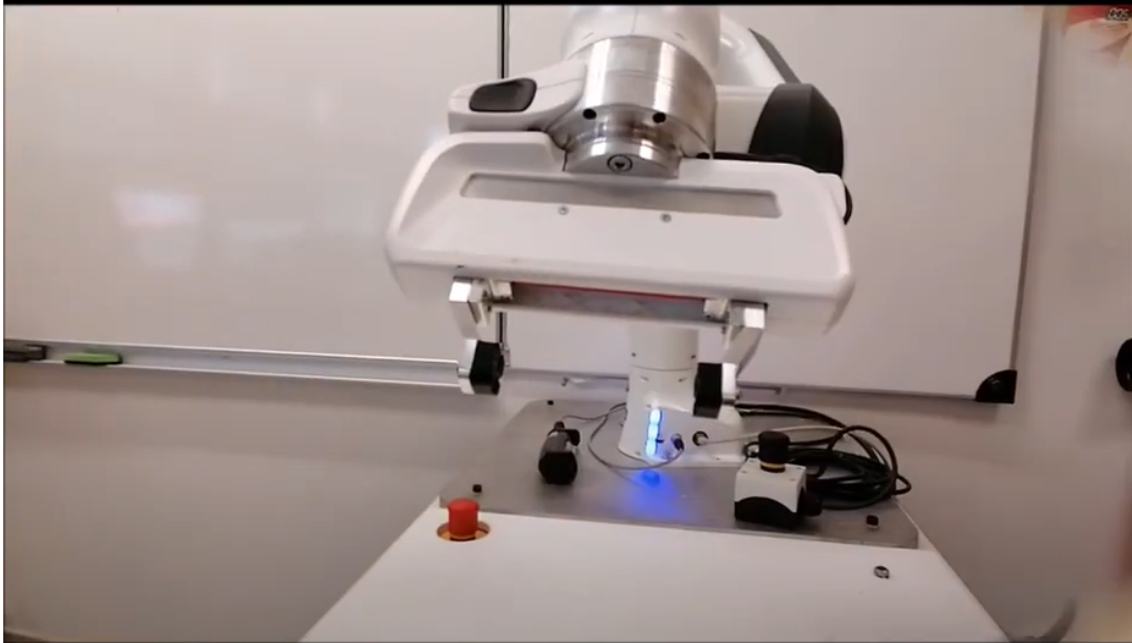


FIGURE 9.4
Print screen of a 2 degrees of freedom gripper taken from [23]

Title	Two degrees of freedom gripper
Description	Frame with 2 "fingers" that close together. The fingers have sponge to adapt to brick shape dynamically and frame can be rotated to face the bricks or to face the storage of the robot.
# of actuators	2
Robustness w.r.t brick position	3 (must be within grippers reach, lateral disposition should be ok within limits)
Robustness w.r.t brick constructions	4 (sponge -> Adaptive to different shapes, higher shapes can be grabbed from bottom, can break during fall)
Robustness w.r.t reliability	5 (apparatus should not break easily, fault detection not too complicated)
Mechanical complexity	2 (main concern is gripper design)
Software complexity	2 (need mechanism to stop gripper motor when object is grabbed)
Speed	3 (3-step process: grab -> move -> release)
Estimated cost	200CHF
Additional comments	-

TABLE 9.4
2 degrees of freedom gripper

9.2 SUCTION MANIPULATOR

Lastly there is the suction manipulator which is similar to the previous gripper but instead of using 2 fingers to grab to the bricks, we would have make use of the flat surfaces of the duplos and used a suction

mechanism.

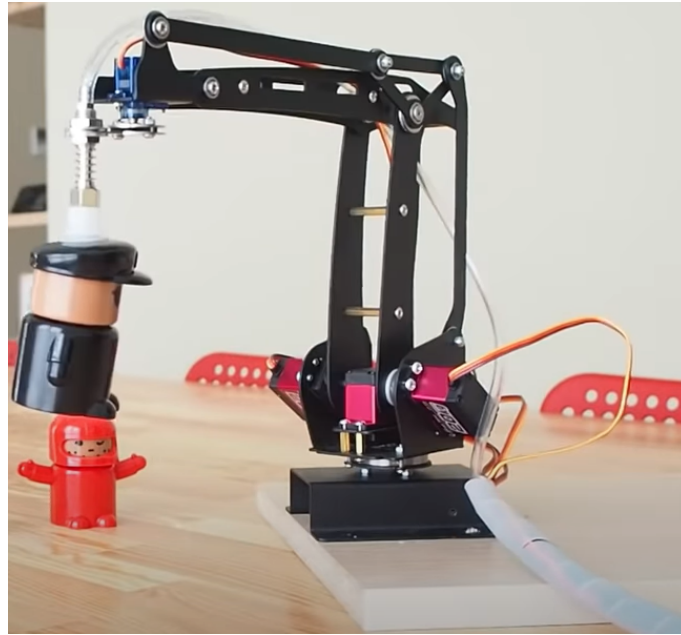


FIGURE 9.5
Suction mechanism system taken from [24]

Title	Suction gripper
Description	Similar to the manipulation arm but instead of having fingers at the end effector we have a suction mechanism.
# of actuators	3-5
Robustness w.r.t brick position	2 (like a classic manipulator but using the fact that bricks have smooth surface and no need to control width of the "fingers")
Robustness w.r.t brick constructions	2 (It is similar to the manipulator but needs less precise positioning of the arm)
Robustness w.r.t reliability	3 (bricks could fall off of the suction cup if not properly placed)
Mechanical complexity	1 (Same concerns as manipulator but needing a tube make things more complicated to mount without leakage)
Software complexity	5 (Same as manipulator although there is less computation since the end effector is simpler)
Speed	2 (same as manipulator, could be faster since no need to adjust the fingers)
Estimated cost	200CHF
Additional comments	-

TABLE 9.5
Suction idea

9.3 FULL NODE GRAPH

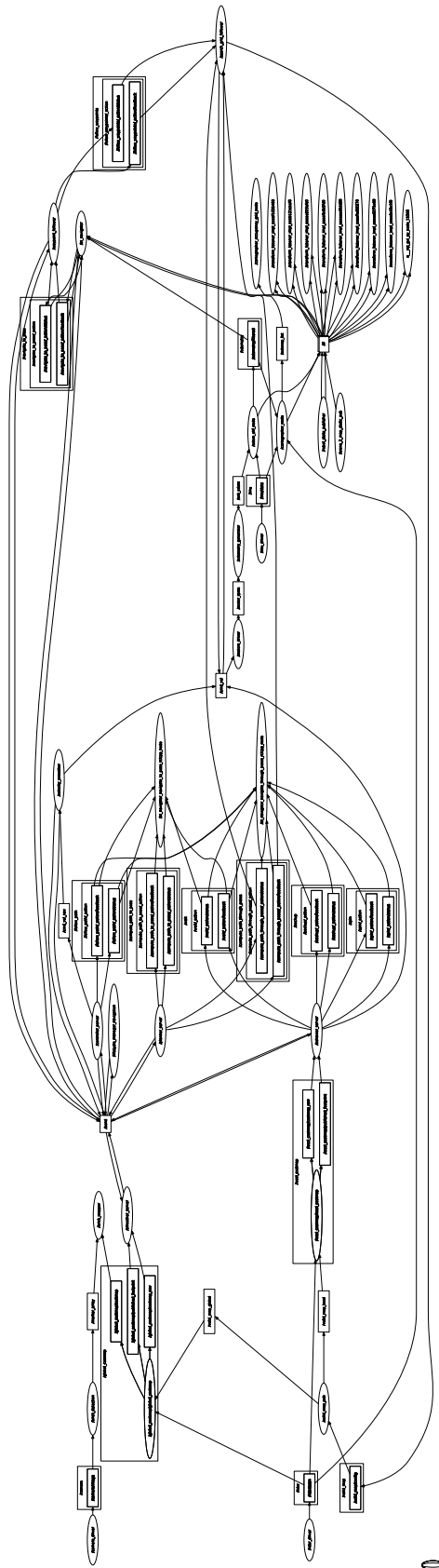


FIGURE 9.6
Full Node Graph

9.4 CRONTAB CONFIGURATION

LISTING 9.1
Root Crontab Configuration

```
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (/tmp/crontab.rh4gPE/crontab installed on Sat Apr 15 12:20:45 2023)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command

@reboot chmod 666 /dev/ttyUSB0
@reboot chmod 666 /dev/i2c-1
@reboot chmod 666 /dev/video0
@reboot pigpiod
@reboot chmod 666 /dev/dma_heap/linux,cma ↵
        /dev/video* /dev/media* /dev/v4l-subdev0
```

9.5 CUSTOM CAMERA CALIBRATION

Camera calibration is performed with a pattern of ArUco markers of increasing index placed in front of the robot (see for example fig. 9.7). The pattern must be aligned with the `base_link`'s x and y axis, i.e. with the drive axle of the robot. When we provide the algorithm the red quantities of the mentioned figure, it can compute the cartesian coordinates of each marker's corners w.r.t the `base_link` frame $(x, y)_{corner,i}$. The corner pixel coordinates $(u, v)_{corner,i}$ can be extracted from the camera image with standard OpenCV operations. The elementwise mapping from extracted pixel-corner to cartesian corner is clear through the indexing of the markers. The final continuous mapping function is found with intercept-enabled linear regression using the C++ Eigen library. The whole calibration process can be

performed in under 5 minutes. The critical points are the correct placement of the pattern in front of the robot and the precise measurement of the marker's extents/distances.

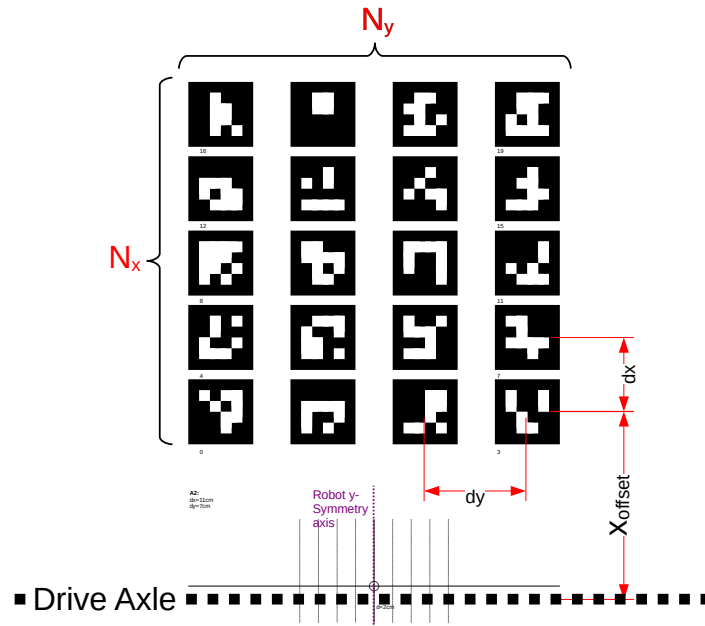


FIGURE 9.7
Required Quantities from Exemplary Calibration Pattern

9.6 DUPLO DETECTION TRAINING SET

Number of Samples	8500
Approximate Number of Adversarial Samples	3400
Average Number of Constructions in Non-Adversarial Samples	5
Sample Resolution	640×640 pixels

TABLE 9.6
Training Dataset Metadata

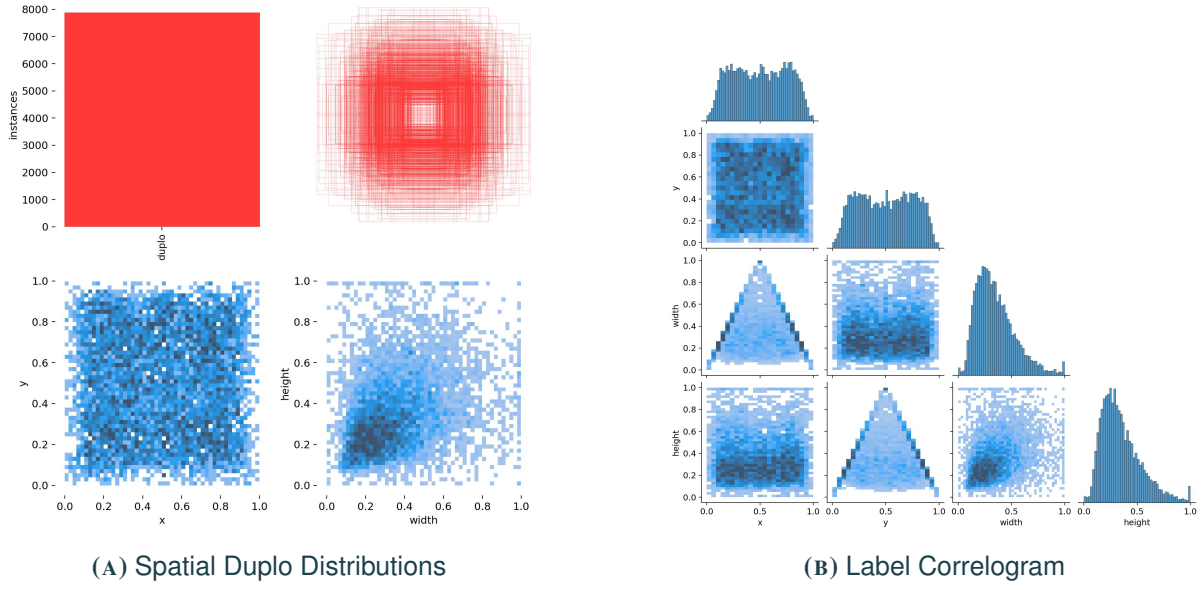


FIGURE 9.8
Training Labels

9.7 TRAINING METRICS

GPU	NVIDIA GeForce GTX 1060 6GB
Datatype	int8
Batch-Size	16
Epochs	4
Total Duration	≈ 20 Minutes
GPU Memory Utilization	≈ 3.8 GB
Validation-Split	5.9%

TABLE 9.7
Training Description

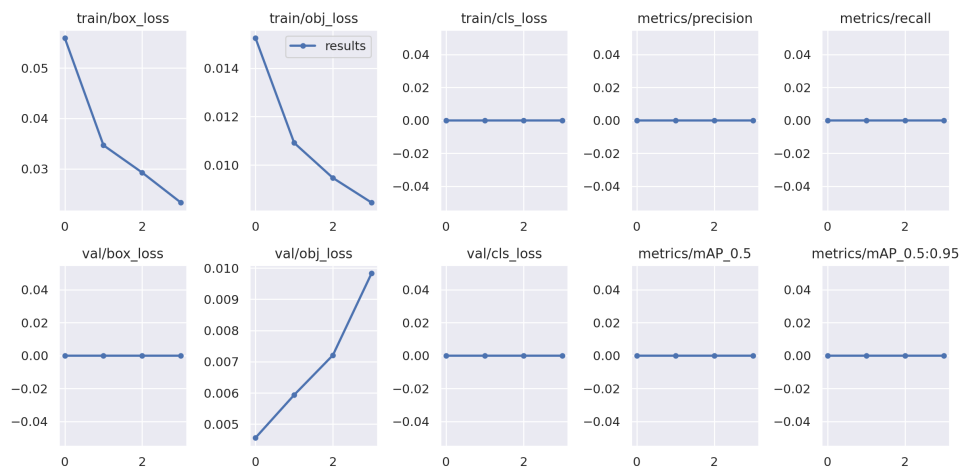


FIGURE 9.9
Loss Evolution

9.8 CUSTOM NAV2 BEHAVIOR TREE

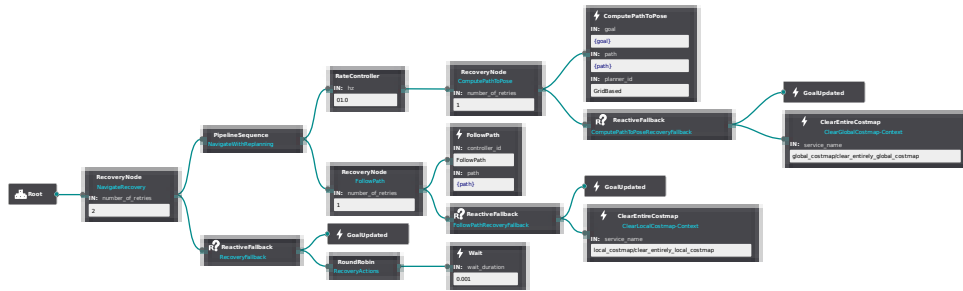


FIGURE 9.10
Custom Behavior Tree

9.9 SEARCH GRID WAYPOINTS

$x[m]$	$y[m]$	yaw $[\circ]$
4.0	1.0	90.0
4.0	1.5	180.0
2.5	1.5	90.0
2.5	3.0	180.0
1.3	3.5	0.0
4.0	3.5	0.0
6.7	3.5	90.0
6.7	4.5	180.0
4.0	4.5	180.0
1.3	4.5	0.0
5.0	4.5	90.0
5.0	7.0	180.0
4.0	7.0	-90.0
4.0	4.5	-90.0
1.0	1.0	-135.0
1.0	1.0	-135.0
1.0	1.0	-135.0

TABLE 9.8
Search Grid Waypoints as `base_link` target coordinates expressed in `arena` frame

9.10 PURCHASE HISTORY

ID	Timestamp	Amount	Description	
2807	24/02/2023 12:26:09	2000.00	Initial budget	Definitive Failure
2897	09/03/2023 16:40:00	-8.00	9 Degrees of Freedom IMU-9150 [#7]	Unused
2898	09/03/2023 16:40:00	-58.25	Raspberry Pi 4 (4 GB) [#7]	Broke through Power Board Failure Broke after wiring error by us
2899	09/03/2023 16:40:00	-68.20	Maxon ESCON 24/2 controller [#30]	
2900	09/03/2023 16:40:00	-12.00	ESCON 24/2 motherboard [#14]	
2901	09/03/2023 16:40:00	-108.90	Maxon EC 32 flat (15 W), 1:60 gearbox [#11]	
2902	09/03/2023 16:40:00	-3.00	Micro-HDMI→HDMI adapter [#9]	
2903	09/03/2023 16:40:00	-21.95	75:1 DC motor (HP) [#13]	
2904	09/03/2023 16:40:00	-5.00	Maxon EC-flat to MM8 adapter [#9]	
2905	09/03/2023 16:40:00	-25.00	Raspberry Pi Camera v2.1 with IR filter [#1]	
2906	09/03/2023 16:40:00	-12.00	ESCON 24/2 motherboard [#9]	
2907	09/03/2023 16:40:00	-108.90	Maxon EC 32 flat (15 W), 1:60 gearbox [#10]	
2908	09/03/2023 16:40:00	-21.95	75:1 DC motor (HP) [#21]	
2909	09/03/2023 16:40:00	-68.20	Maxon ESCON 24/2 controller [#29]	Broke after wiring error by us
2910	09/03/2023 16:40:00	-12.25	MicroSD card 32 GB UHS-I [#9]	
2911	09/03/2023 16:40:00	-5.00	Maxon EC-flat to MM8 adapter [#6]	
2955	30/03/2023 12:57:18	68.20	Return: Maxon ESCON 24/2 controller [#29]	
2956	30/03/2023 12:58:02	-68.20	Maxon ESCON 24/2 controller [#28]	
2957	31/03/2023 13:07:10	-68.20	Maxon ESCON 24/2 controller [#14]	
2958	31/03/2023 13:07:10	-7.50	WildTumper 120x60 mm wheel, 4 mm shaft [#1]	Broke after charging with balancing cable in wrong direction
2959	31/03/2023 13:07:10	-40.00	LiPo Battery 3S (11.1 V) 4000 mAh [#2]	Broke of unknown reason
2960	31/03/2023 13:07:10	-25.00	LiPo protection & power board [#14]	
2961	31/03/2023 13:07:10	-40.00	LiPo Battery 3S (11.1 V) 4000 mAh [#11]	
2962	31/03/2023 13:07:10	-7.50	WildTumper 120x60 mm wheel, 4 mm shaft [#2]	
2971	05/04/2023 13:30:21	-4.00	Flex cable for Raspberry Pi camera - 1m [#2]	
2985	18/04/2023 15:13:40	-5.00	Reely servo S-0008	
3018	26/04/2023 14:00:46	5.00	Return: Reely servo S-0008	
3019	26/04/2023 14:05:46	-7.00	Parallax standard servo [#4]	Were broken already (taken from untested materials)
3020	26/04/2023 14:05:46	-7.00	Parallax standard servo [#2]	
3040	10/05/2023 15:03:51	-9.95	100mm mud country dick cepek wheel [#2]	
3041	10/05/2023 15:13:42	-15.00	110mm Chainlink AKA14017 wheel [#2]	
3042	10/05/2023 15:13:42	-15.00	110mm Chainlink AKA14017 wheel [#1]	
3063	24/05/2023 12:18:14	-23.00	Servomotor Master DS8020 (13 kg) [#1]	
3064	24/05/2023 12:18:14	-23.00	Servomotor Master DS8020 (13 kg) [#8]	
3065	24/05/2023 12:18:14	-23.00	Servomotor Master DS8020 (13 kg) [#4]	
3081	08/06/2023 15:16:41	-40.00	LiPo Battery 3S (11.1 V) 4000 mAh [#13]	
3082	09/06/2023 11:06:58	-25.00	LiPo protection & power board [#18]	

FIGURE 9.11
Purchases made with Virtual Budget and Status of Components

ID	Timestamp	Amount	Online Shop	Content
2913	15/03/2023 09:50:19	-93.80	maker-shop.ch	Coral USB AI Accelerator
2920	17/03/2023 14:41:04	-135.46	reichelt.ch	RPLIDAR A1M8
3044	15/05/2023 12:03:31	-161.60	ch.rs-online.com/web	Belt Components, Alu Profiles

FIGURE 9.12
Purchases made in Online Shops

Item	Exact Description	Shop	Price in CHF
Small Castor Wheel	Wagner roulette pivo t. d30mm.pu.cap.15kg	Jumbo Crissier	3.75
Large Castor Wheel	Wagner roulette pivo t. d50mm.pu.cap.50kg	Jumbo Crissier	7.25
Aluminium Bars (×3)	BA-Profil Vierkant Natur 15 x 15 x 1'000 mm	OBI Renens	16.50
Cylindric Foam	Traversin	Jumbo Crissier	10.50

TABLE 9.9
Purchases made in Local Hardware Shops

9.11 GANTT CHART OF MIDTERM PRESENTATION

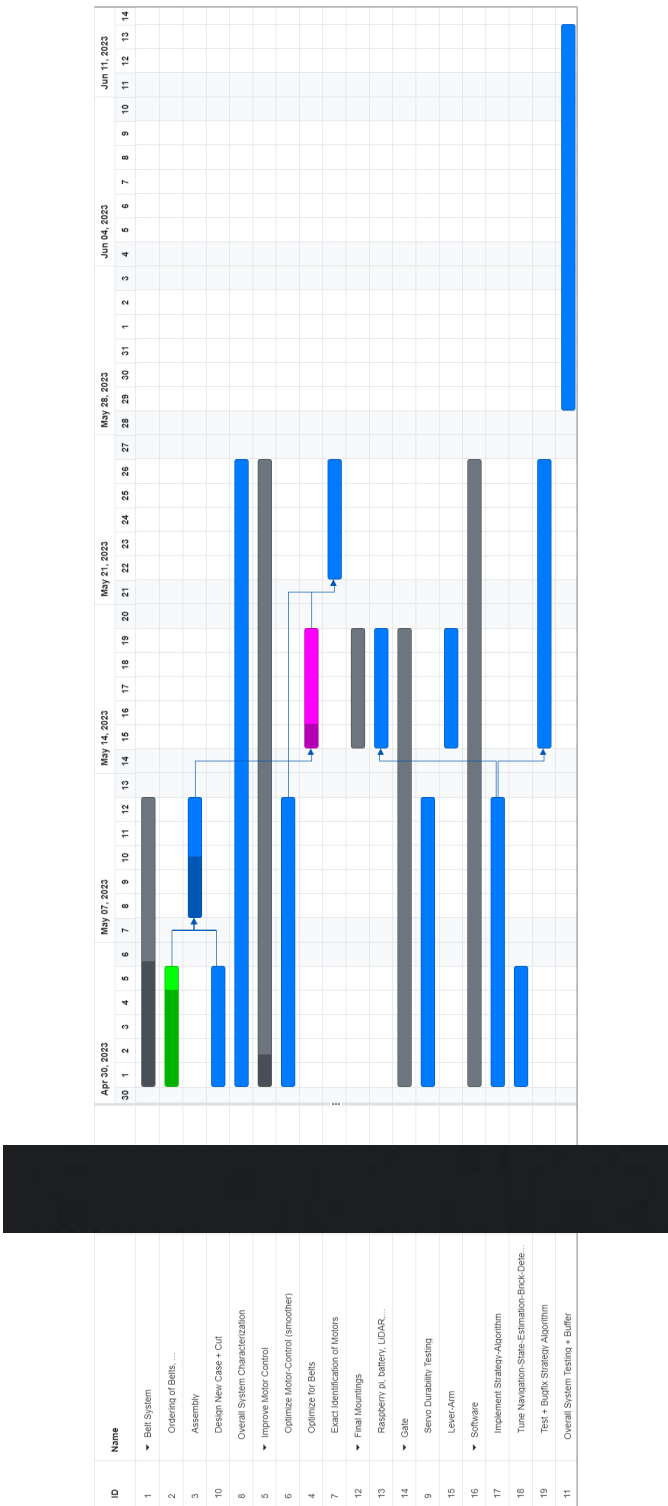
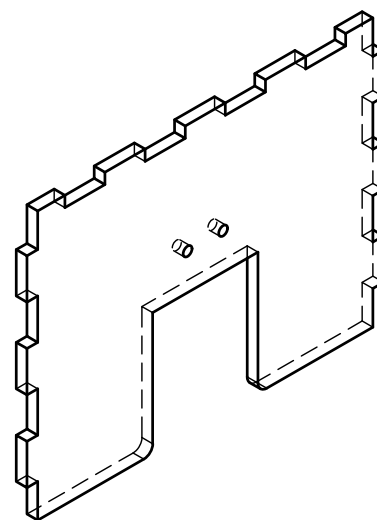
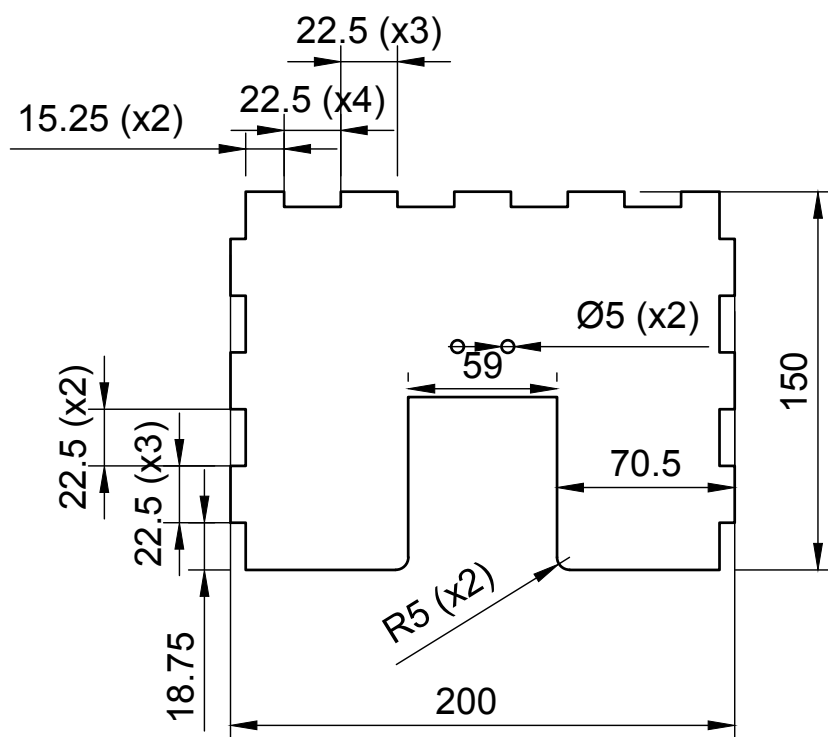
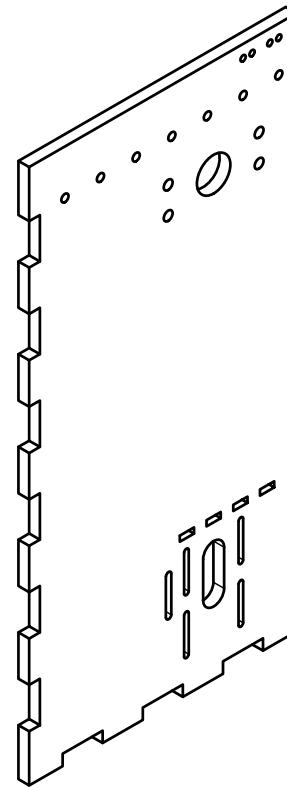
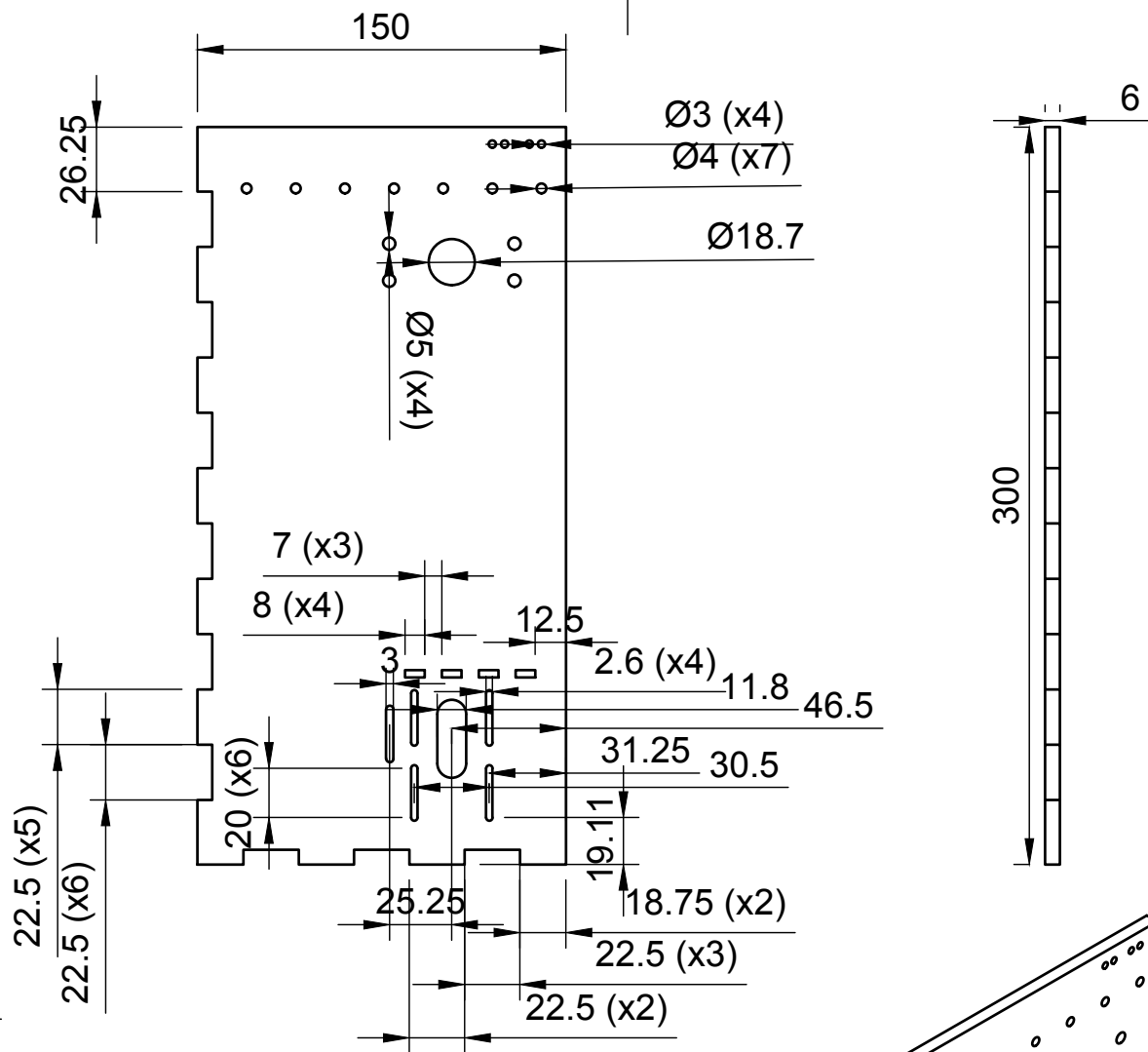


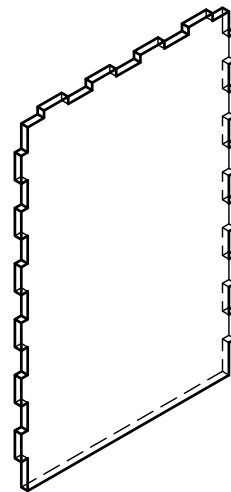
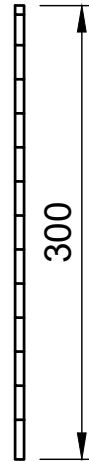
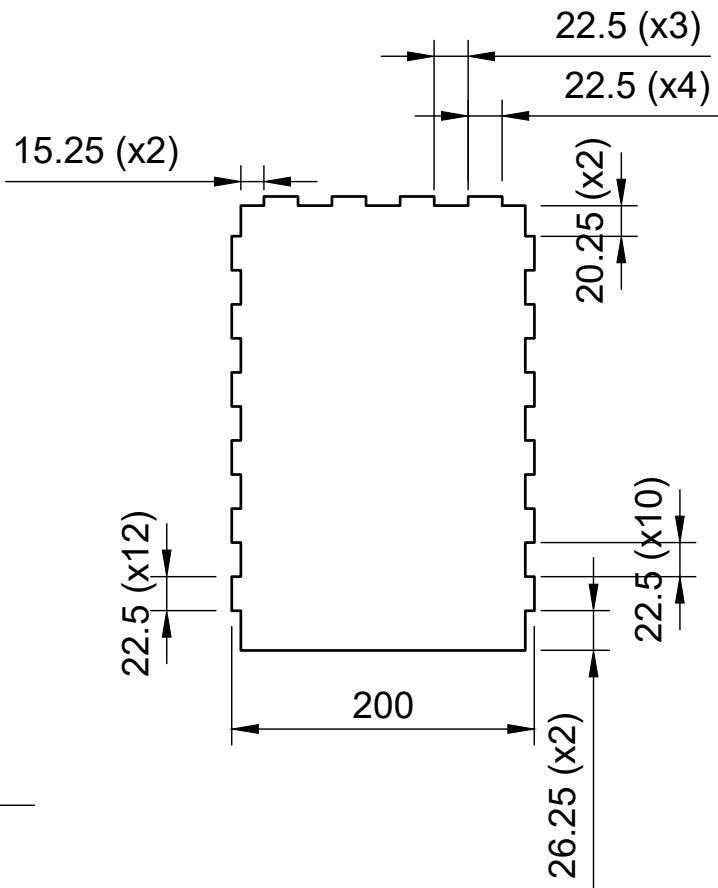
FIGURE 9.13
Gantt Chart of Midterm Presentation



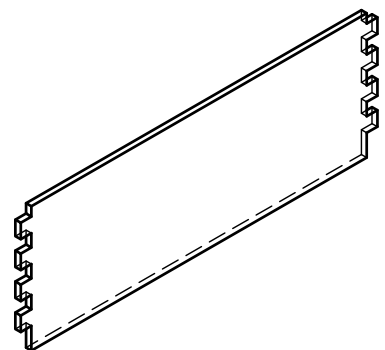
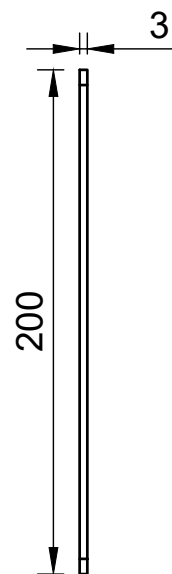
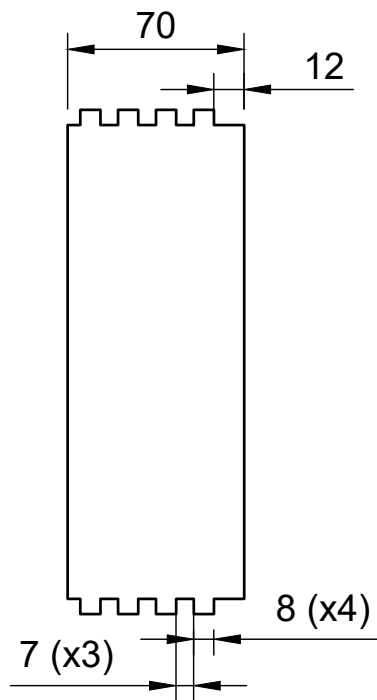
Dept.	Technical reference	Approved by		
		Document type	Document status	
		Title	DWG No.	
		backplate	Rev.	Date of issue
			Sheet	1/1



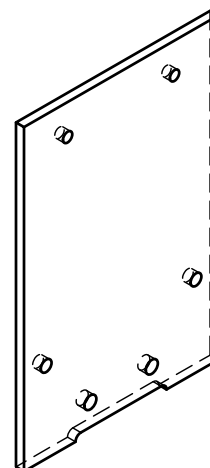
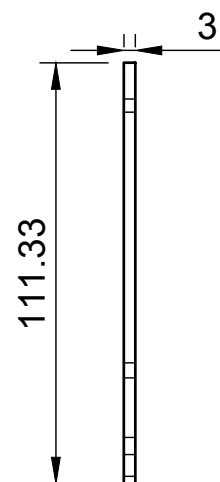
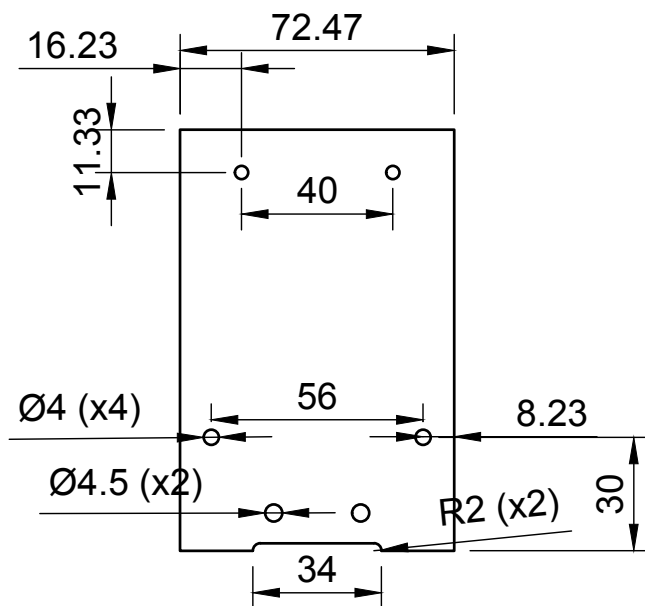
Dept.	Technical reference		Approved by			
		Document type	Document status			
		Title	DWG No.			
		Side plate	Rev.	Date of issue	Sheet	
					1/1	



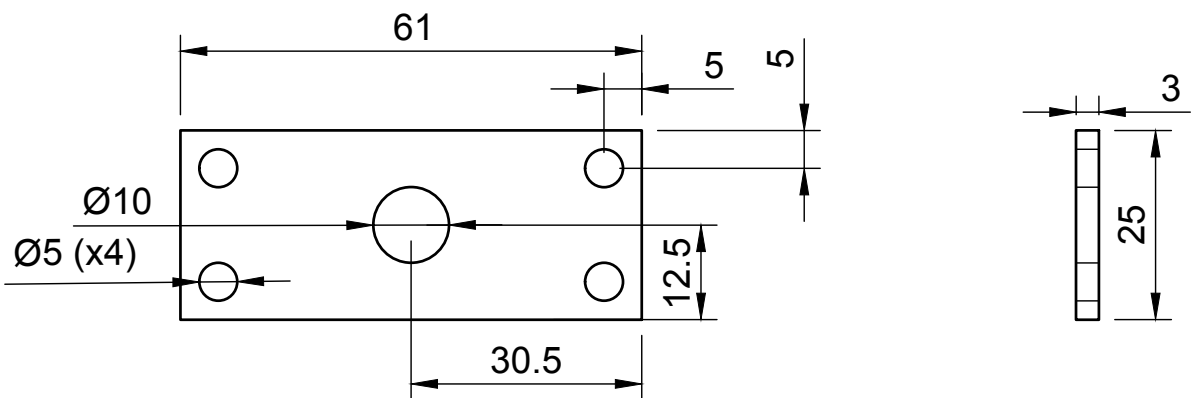
Dept.	Technical reference		Approved by		
		Document type	Document status		
		Title	DWG No.		
		top plate	Rev.	Date of issue	Sheet
					1/1



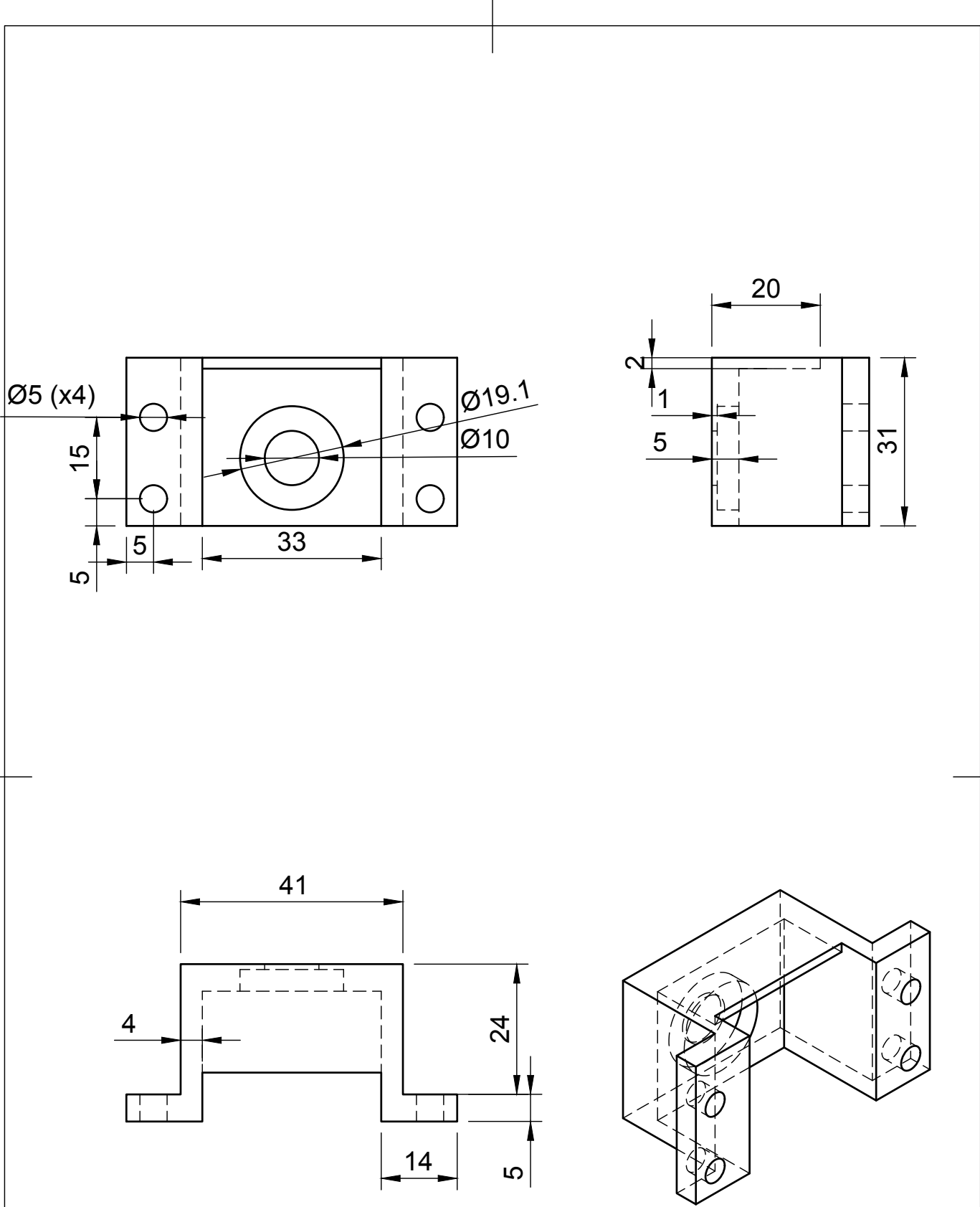
Dept.	Technical reference				Approved by
		Document type	Document status		
		Title	DWG No.		
		motor_protec	Rev.	Date of issue	Sheet
					1/1



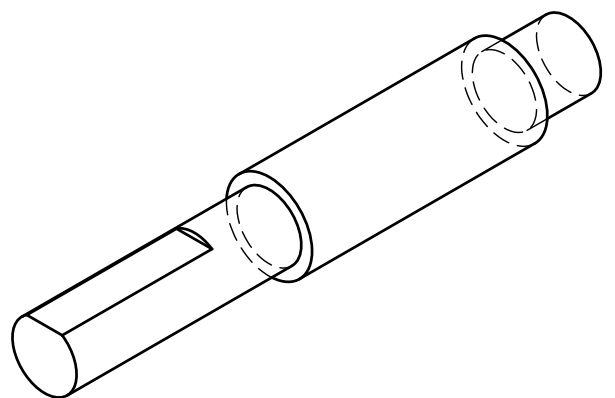
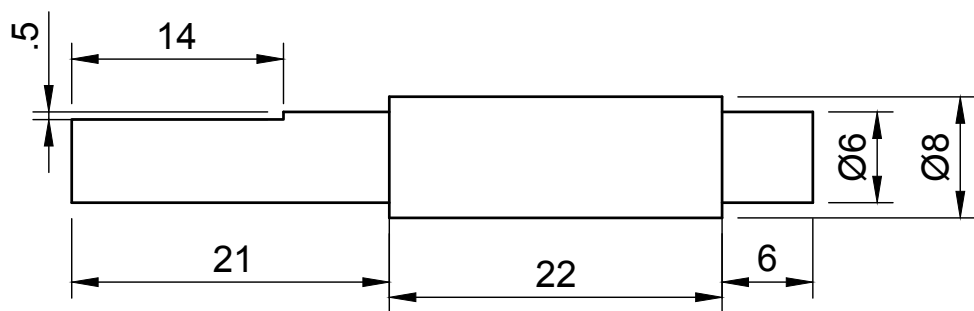
Dept.	Technical reference		Approved by
		Document type	Document status
		Title	DWG No.
		LIDAR_plate	
Rev.	Date of issue	Sheet	
		1/1	



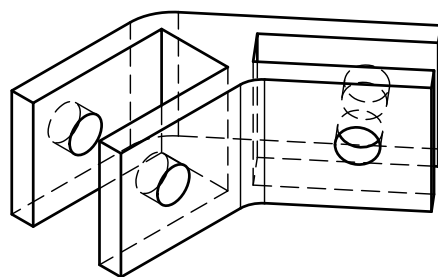
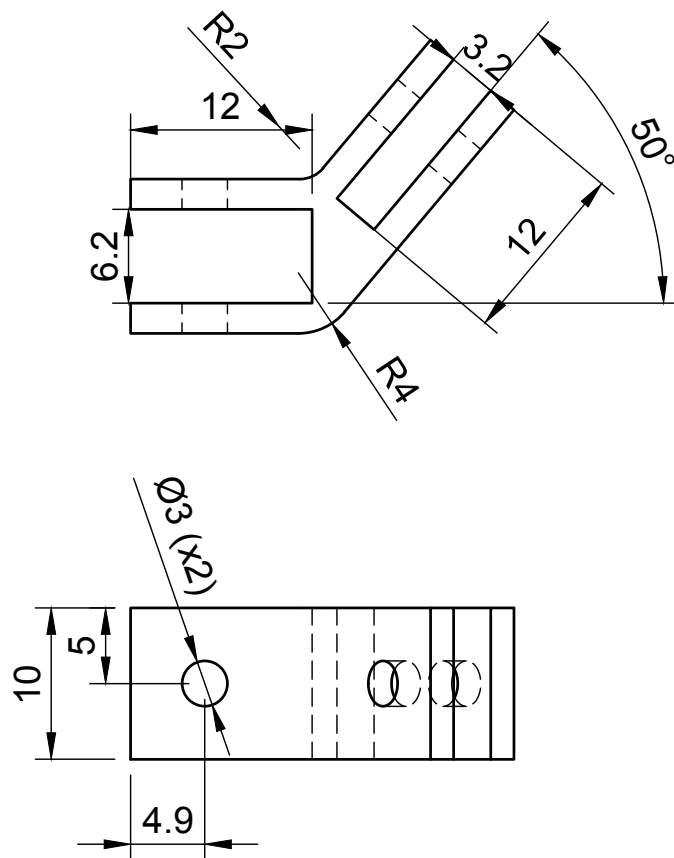
Dept.	Technical reference				Approved by
		Document type	Document status		
		Title	DWG No.		
		wheel mount_2	Rev.	Date of issue	Sheet
					1/1



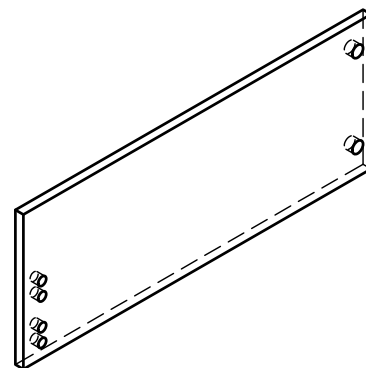
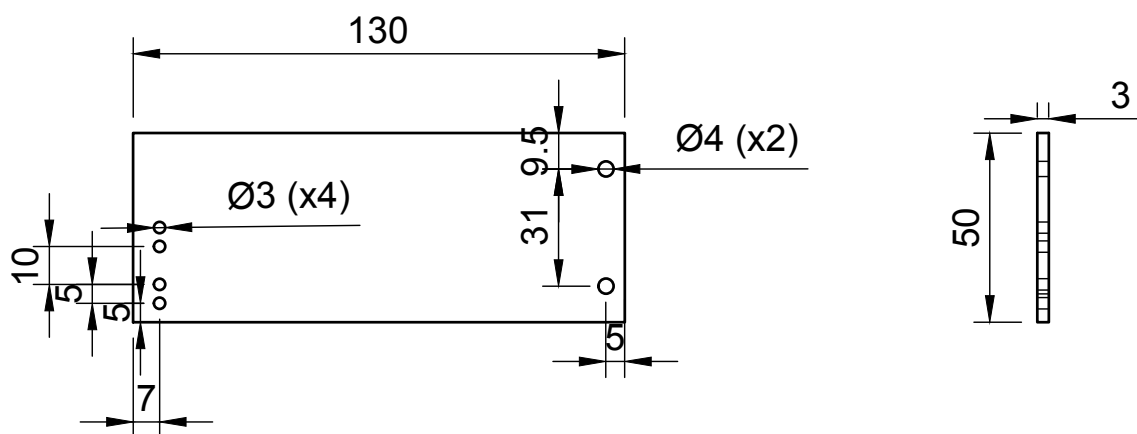
Dept.	Technical reference	Approved by		
		Document type	Document status	
		Title	DWG No.	
		Rev.	Date of issue	Sheet
				1/1



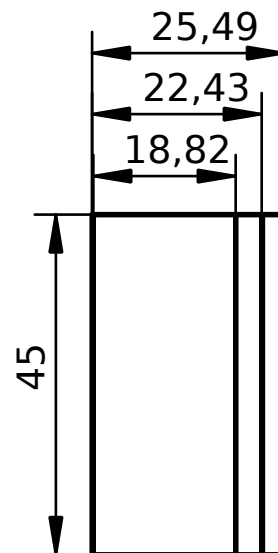
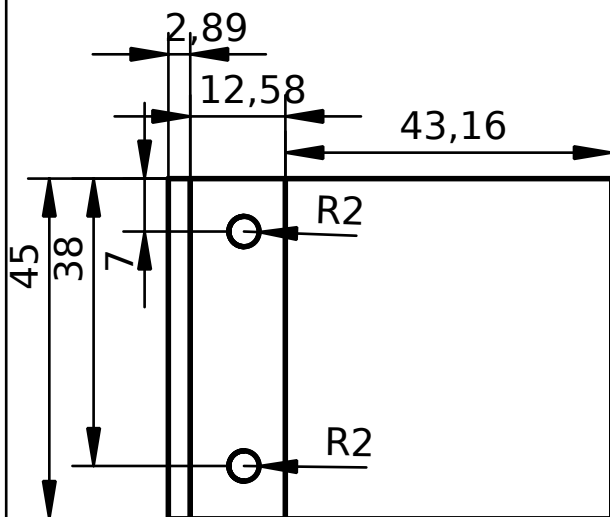
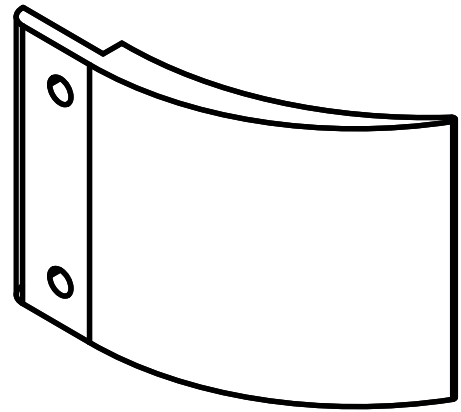
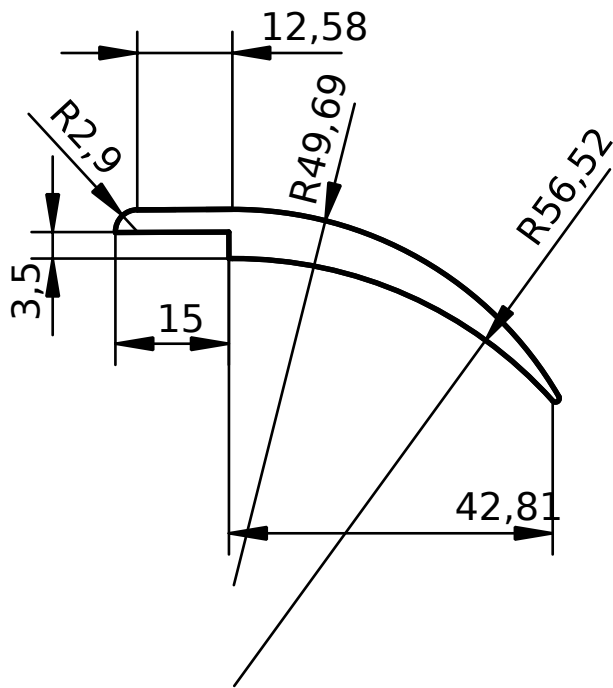
Dept.	Technical reference		Approved by		
		Document type	Document status		
		Title wheel_axle	DWG No.		
			Rev.	Date of issue	Sheet
					1/1



Dept.	Technical reference		Approved by		
		Document type	Document status		
		Title	DWG No.		
		funnel fixation	Rev.	Date of issue	Sheet
					1/1



Dept.	Technical reference				Approved by
		Document type	Document status		
		Title	DWG No.		
		funnelboard	Rev.	Date of issue	Sheet
					1/1



Created by:

Title:

Flexible Fangs for Funnel

Supplementary information:

Robot Competition - Team 2

Size:

A4

Sheet:

1 / 1

Scale:

1:1

Part number:

-

Drawing number:

-

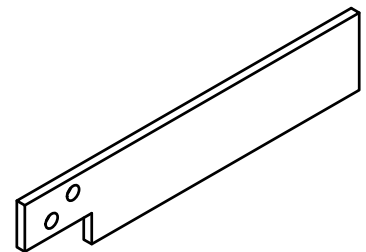
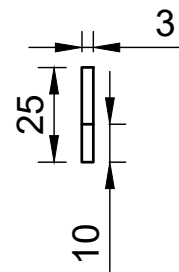
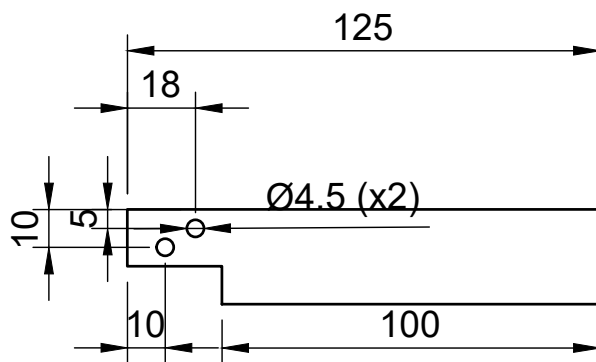
Date:

13/06/2023

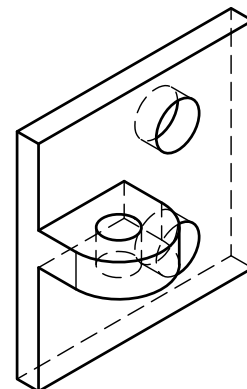
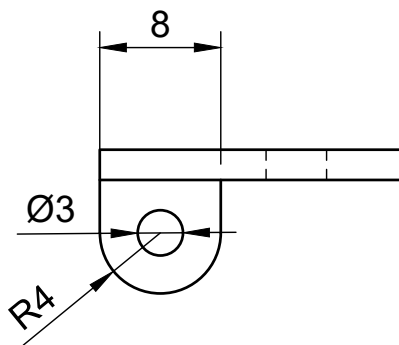
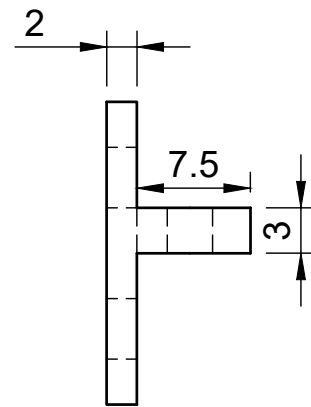
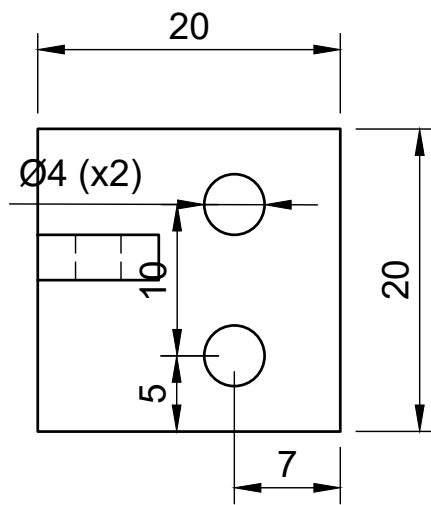
Revision:

REV A

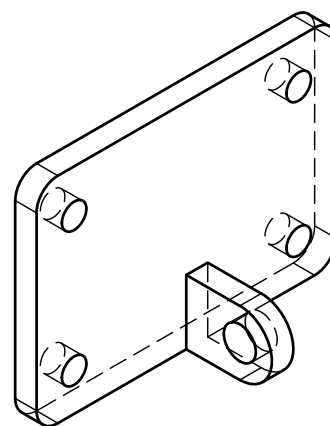
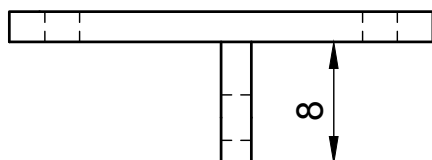
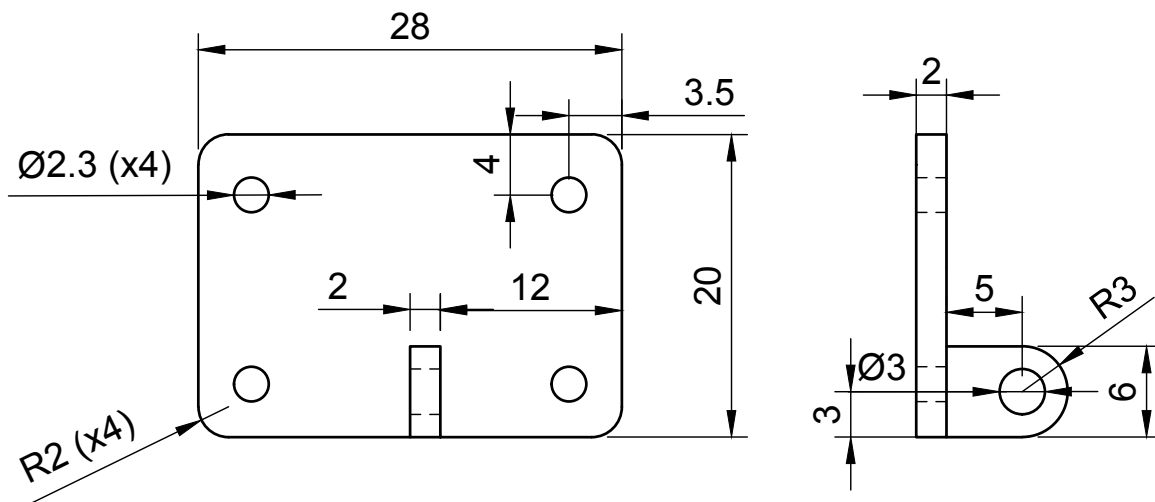




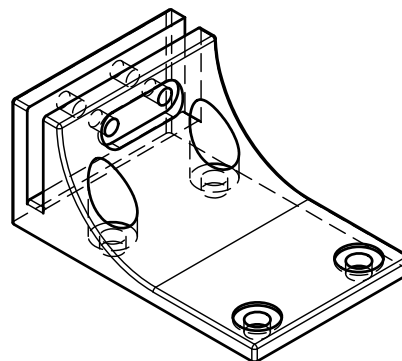
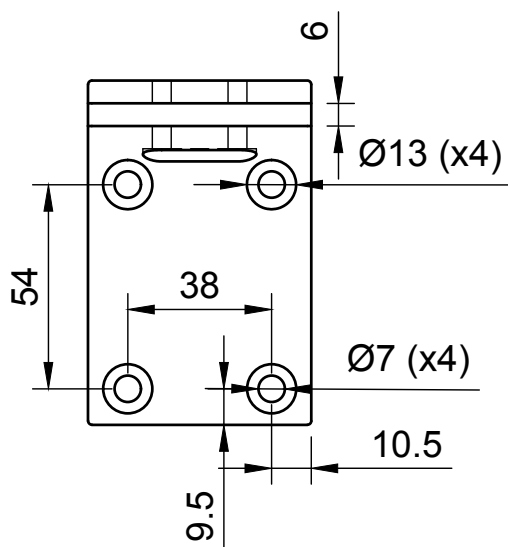
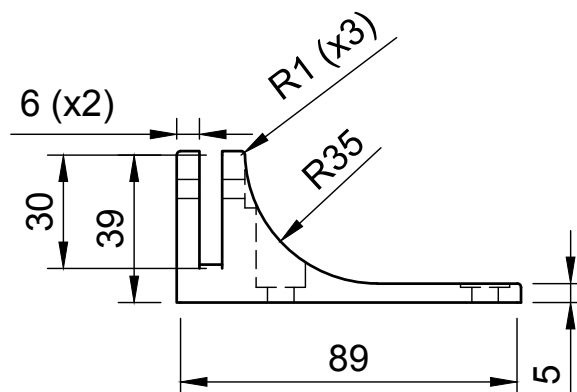
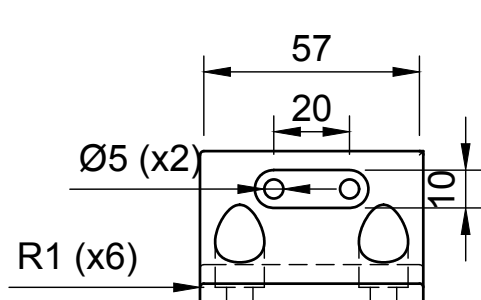
Dept.	Technical reference	[REDACTED]			Approved by
		Document type	Document status		
		Title	DWG No.		
		wheel guard backside	Rev.	Date of issue	Sheet
					1/1



Dept.	Technical reference		Approved by			
		Document type	Document status			
		Title	DWG No.			
		Camera mount profile side	Rev.	Date of issue	Sheet	
					1/1	



Dept.	Technical reference	Approved by		
		Document type	Document status	
		Title	DWG No.	
		mount_camera_side	Rev.	Date of issue
			Sheet	1/1



Dept.	Technical reference	Approved by		
		Document type	Document status	
		Title	DWG No.	
		mount_castor big	Rev.	Date of issue
				Sheet
				1/1

BIBLIOGRAPHY

- [1] *Robot competition rule book*. 2023. URL: <https://robot-competition.epfl.ch/download/rulebook.pdf>.
- [2] Helmut Neukirchen. *Power consumption of Raspberry Pi 4 versus Intel J4105 system*. 2021. URL: <https://uni.hi.is/helmut/2021/06/07/power-consumption-of-raspberry-pi-4-versus-intel-j4105-system/>.
- [3] Maxon. *EC 32 flat 32 mm, brushless, 15 watt*. 2021. URL: https://www.maxongroup.com/medias/sys_master/root/8882562662430/EN-21-292.pdf.
- [4] Maxon. *ESCON Module 24/2*. 2018. URL: https://www.maxongroup.com/medias/sys_master/root/8834400092190/466023-ESCON-Module-24-2-Hardware-Reference-En.pdf.
- [5] coral.ai. *USB Accelerator datasheet*. 2019. URL: <https://coral.ai/static/files/Coral-USB-Accelerator-datasheet.pdf>.
- [6] Raspberry Pi Ltd. *Raspberry Pi - camera, reducing power consumption*. 2016. URL: <https://forums.raspberrypi.com/viewtopic.php?t=152864#:~:text=The%5C%20camera%5C%20takes%5C%20about%5C%20200,can%5C%20do%5C%20to%5C%20change%5C%20that..>
- [7] reichelt. *RPLIDAR A1M8 RPLIDAR A1M8 360° Laser Scanner 12m*. 2023. URL: https://www.reichelt.com/ch/de/rplidar-alm8-360-laser-scanner-12m-rplidar-alm8-p314357.html?&trstct=pos_15&nbc=1.
- [8] Frank Dellaert and Seth Hutchinson. *Motion Model for the Differential Drive Robot*. 2021. URL: https://www.roboticsbook.org/S52_diffdrive_actions.html.
- [9] Mario Garcia. *Quaternion Integration*. 2023. URL: <https://ahrs.readthedocs.io/en/latest/filters/angular.html#quaternion-integration>.
- [10] *Building libcamera and libcamera-apps*. 2023. URL: https://www.raspberrypi.com/documentation/computers/camera_software.html#building-libcamera-and-libcamera-apps.
- [11] kbarni. *libcamera bindings for OpenCV*. 2022. URL: <https://github.com/kbarni/LCCV>.
- [12] *Get started with the USB Accelerator*. 2020. URL: <https://coral.ai/docs/accelerator/get-started/>.
- [13] *Build TensorFlow Lite Python Wheel Package*. 2020. URL: https://www.tensorflow.org/lite/guide/build_cmake_pip.
- [14] *robot_localizationpackage for ROS2*. 2023. URL: https://github.com/cra-ros-pkg/robot_localization.
- [15] Wolfgang Hess et al. 'Real-time loop closure in 2D LIDAR SLAM'. In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278. DOI: 10.1109/ICRA.2016.7487258.
- [16] *Nav2*. 2020. URL: <https://navigation.ros.org/>.

- [17] Steve Macenski et al. *Regulated Pure Pursuit for Robot Path Tracking*. 2023. arXiv: 2305.20026 [cs.RO].
- [18] *YOLOv5 in PyTorch > ONNX > CoreML > TFLite*. 2023. URL: <https://github.com/ultralytics/yolov5>.
- [19] *Doblo factory*. 2021. URL: https://edutechwiki.unige.ch/en/Doblo_factory.
- [20] *LEGO Catalog Database Download*. 2023. URL: <https://rebrickable.com/downloads/>.
- [21] Péter Fankhauser and Marco Hutter. ‘A Universal Grid Map Library: Implementation and Use Case for Rough Terrain Navigation’. In: *Robot Operating System (ROS) – The Complete Reference (Volume 1)*. Ed. by Anis Koubaa. Springer, 2016. Chap. 5. ISBN: 978-3-319-26052-5. DOI: 10.1007/978-3-319-26054-9{_}5. URL: <http://www.springer.com/de/book/9783319260525>.
- [22] *Galaxus*. 2023. URL: <https://www.galaxus.ch/de/s1/product/joy-it-grab-it-roboterarm-inkl-motorsteuerung-robotik-kit-12900268>.
- [23] Pierre Bérioux. *Utilisation de la fonction GRIPPER MOVE robot FRANKA EMIKA*. 2021. URL: https://www.youtube.com/watch?v=FGXgvVz8_qk&t=45s.
- [24] Maker 101. *Vacuum Suction Robot Arm*. 2020. URL: <https://youtu.be/eVz7mOJVvQc>.